LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5

CONNECT TO THE INTERNET

Oleh:

ERISCHA MARSELA **NIM. 2410817120022**

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect To The Internet Sederhana ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman II. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : ERISCHA MARSELA
NIM : 2410817120022

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan	Irham Maulani Abdul Gani, S.Kom.,
NIM. 2310817210027	M.Kom.
	NIP. 199710312025061009

DAFTAR ISI

DAFTAR TABEL

DAFTAR GAMBAR

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
b. Gunakan KotlinX Serialization sebagai library JSON.
c. Gunakan library seperti Coil atau Glide untuk image loading.
d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: https://developer.themoviedb.org/docs/getting-started e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
e. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
f. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. OUTPUT CODE

Tabel 1. Source Code MainActivity.kt

```
1.  package com.example.clickanime
2.
3.  import android.content.Context
4.  import android.os.Bundle
5.  import androidx.activity.ComponentActivity
6.  import androidx.activity.compose.setContent
7.  import androidx.activity.enableEdgeToEdge
8.  import
    androidx.compose.foundation.layout.fillMaxSize
9.  import androidx.compose.material3.Surface
10. import androidx.compose.runtime.remember
11. import androidx.compose.ui.Modifier
```

```
12. import androidx.lifecycle.viewmodel.compose.viewModel
13. import androidx.navigation.compose.rememberNavController
14. import com.example.clickanime.navigation.NavGraph
15. import com.example.clickanime.ui.theme.MovieAppTheme
16. import com.example.clickanime.viewmodel.MovieViewModel
17. import java.util.Locale
18.
19. class MainActivity : ComponentActivity() {
20.
21.     override fun attachBaseContext(newBase: Context) {
22.         val prefs = newBase.getSharedPreferences("clickanime_settings", Context.MODE_PRIVATE)
23.         val lang = prefs.getString("language", Locale.getDefault().language) ?: "en"
24.         val locale = Locale(lang)
25.         Locale.setDefault(locale)
26.         val config = newBase.resources.configuration
27.         config.setLocale(locale)
28.
super.attachBaseContext(newBase.createConfigurationContext(config))
29.     }
30.
31.     override fun onCreate(savedInstanceState: Bundle?) {
32.         super.onCreate(savedInstanceState)
33.         enableEdgeToEdge()
34.
35.         val app = application as MovieApp
36.
37.         setContent {
```

```
38.             MovieAppTheme {
39.                 Surface(modifier = Modifier.fillMaxSize()) {
40.                     val navController = rememberNavController()
41.                     val movieViewModel: MovieViewModel = viewModel(
42.                         factory = app.container.movieViewModelFactory
43.                     )
44.                     NavGraph(
45.                         navController = navController,
46.                         movieViewModel = movieViewModel,
47.                         appPreferences = app.container.appPreferences,
48.                         onLocaleChange = { langCode -> applyLocale(langCode) }
49.                     )
50.                 }
51.             }
52.         }
53.     }
54.
55.     private fun applyLocale(langCode: String) {
56.         getSharedPreferences("clickanime_settings", Context.MODE_PRIVATE)
57.             .edit()
58.             .putString("language", langCode)
59.             .apply()
60.         recreate()
61.     }
62. }
```

Tabel 2. Source Code MovieApp.kt

```
1.  package com.example.clickanime
2.
3.  import android.app.Application
4.  import com.example.clickanime.di.AppContainer
5.  import timber.log.Timber
6.
7.  class MovieApp : Application() {
8.
9.      lateinit var container: AppContainer
10.         private set
11.
12.     override fun onCreate() {
13.         super.onCreate()
14.         container = AppContainer(this)
15.         if (BuildConfig.DEBUG) {
16.             Timber.plant(Timber.DebugTree())
17.         }
18.         Timber.d("MovieApp initialized")
19.     }
20. }
```

Tabel 3. Source Code MovieDao.kt

```
1.  package com.example.clickanime.data.local.dao
2.
3.  import androidx.room.*
4.  import
    com.example.clickanime.data.local.entity.FavoriteMov
    ieEntity
5.  import
    com.example.clickanime.data.local.entity.MovieDetail
    Entity
6.  import
    com.example.clickanime.data.local.entity.MovieEntity
```

```
import kotlinx.coroutines.flow.Flow


@Dao
interface MovieDao {

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertMovies(movies: List<MovieEntity>)

    @Query("SELECT * FROM movies WHERE category = :category ORDER BY popularity DESC")
    fun getMoviesByCategory(category: String): Flow<List<MovieEntity>>

    @Query("SELECT MAX(cachedAt) FROM movies WHERE category = :category")
    suspend fun getLatestCacheTime(category: String): Long?

    @Query("SELECT * FROM movies WHERE title LIKE '%' || :q || '%' OR originalTitle LIKE '%' || :q || '%'")
    fun searchMovies(q: String): Flow<List<MovieEntity>>

    @Query("DELETE FROM movies WHERE category = :category")
    suspend fun deleteByCategory(category: String)

    @Query("DELETE FROM movies")
    suspend fun clearAll()

    @Query("DELETE FROM movies WHERE cachedAt < :expiryTime")
    suspend fun deleteExpired(expiryTime: Long)
}

```

```
@Dao
interface MovieDetailDao {

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(detail: MovieDetailEntity)

    @Query("SELECT * FROM movie_details WHERE id = :movieId LIMIT 1")
    suspend fun getById(movieId: Int): MovieDetailEntity?

    @Query("SELECT cachedAt FROM movie_details WHERE id = :movieId LIMIT 1")
    suspend fun getCacheTime(movieId: Int): Long?

    @Query("DELETE FROM movie_details WHERE cachedAt < :expiryTime")
    suspend fun deleteExpired(expiryTime: Long)

    @Query("DELETE FROM movie_details")
    suspend fun clearAll()
}

@Dao
interface FavoriteMovieDao {

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun add(favorite: FavoriteMovieEntity)

    @Query("DELETE FROM favorite_movies WHERE movieId = :movieId")
    suspend fun removeById(movieId: Int)

    @Query("SELECT * FROM favorite_movies ORDER BY addedAt DESC")
```

```
63.     fun getAll(): Flow<List<FavoriteMovieEntity>>
64.
65.     @Query("SELECT EXISTS(SELECT 1 FROM
    favorite_movies WHERE movieId = :movieId)")
66.     fun isFavorite(movieId: Int): Flow<Boolean>
67.
68.     @Query("SELECT EXISTS(SELECT 1 FROM
    favorite_movies WHERE movieId = :movieId)")
69.     suspend fun isFavoriteOnce(movieId: Int): Boolean
70.
71.     @Query("SELECT COUNT(*) FROM favorite_movies")
72.     fun count(): Flow<Int>
73. }
```

Tabel 4. Source Code MovieEntity.kt

```
1.  package com.example.clickanime.data.local.entity
2.
3.  import androidx.room.Entity
4.  import androidx.room.PrimaryKey
5.
6.  @Entity(tableName = "movies")
7.  data class MovieEntity(
8.      @PrimaryKey val id: Int,
9.      val title: String,
10.     val originalTitle: String,
11.     val overview: String,
12.     val posterPath: String?,
13.     val backdropPath: String?,
14.     val voteAverage: Double,
15.     val voteCount: Int,
16.     val releaseDate: String,
17.     val genreIds: String,
18.     val popularity: Double,
```

```
    val originalLanguage: String,
    val category: String,
    val cachedAt: Long = System.currentTimeMillis()
)

@Entity(tableName = "movie_details")
data class MovieDetailEntity(
    @PrimaryKey val id: Int,
    val title: String,
    val originalTitle: String,
    val overview: String,
    val posterPath: String?,
    val backdropPath: String?,
    val voteAverage: Double,
    val voteCount: Int,
    val releaseDate: String,
    val runtime: Int?,
    val genres: String,
    val tagline: String,
    val status: String,
    val popularity: Double,
    val budget: Long,
    val revenue: Long,
    val homepage: String,
    val originalLanguage: String,
    val cachedAt: Long = System.currentTimeMillis()
)

@Entity(tableName = "favorite_movies")
data class FavoriteMovieEntity(
    @PrimaryKey val movieId: Int,
    val title: String,
```

```
51.     val posterPath: String?,
52.     val voteAverage: Double,
53.     val releaseDate: String,
54.     val overview: String,
55.     val addedAt: Long = System.currentTimeMillis()
56. )
```

Tabel 5. Source Code AppPreferences.kt

```
1.  package com.example.clickanime.data.local
2.
3.  import android.content.Context
4.
5.  class AppPreferences(context: Context) {
6.
7.      private val prefs = context.getSharedPreferences(PREFS_NAME,
        Context.MODE_PRIVATE)
8.
9.      var language: String
10.         get() = prefs.getString(KEY_LANGUAGE, "en") ?: "en"
11.         set(v) = prefs.edit().putString(KEY_LANGUAGE, v).apply()
12.
13.     var isNotificationsEnabled: Boolean
14.         get() = prefs.getBoolean(KEY_NOTIFICATIONS, false)
15.         set(v) = prefs.edit().putBoolean(KEY_NOTIFICATIONS, v).apply()
16.
17.     var isGridView: Boolean
18.         get() = prefs.getBoolean(KEY_GRID_VIEW, false)
19.         set(v) = prefs.edit().putBoolean(KEY_GRID_VIEW, v).apply()
```

```
20.
21.     var lastMovieCategory: String
22.         get()   =   prefs.getString(KEY_LAST_CATEGORY,
    CAT_POPULAR) ?: CAT_POPULAR
23.         set(v)                                          =
    prefs.edit().putString(KEY_LAST_CATEGORY, v).apply()
24.
25.     companion object {
26.         const val PREFS_NAME   = "clickanime_settings"
27.         private  const  val  KEY_LANGUAGE             =
    "language"
28.         private   const   val   KEY_NOTIFICATIONS     =
    "notifications_enabled"
29.         private  const  val  KEY_GRID_VIEW            =
    "grid_view"
30.         private   const   val   KEY_LAST_CATEGORY     =
    "last_movie_category"
31.
32.         const val CAT_POPULAR     = "popular"
33.         const val CAT_NOW_PLAYING = "now_playing"
34.         const val CAT_TOP_RATED   = "top_rated"
35.         const val CAT_UPCOMING    = "upcoming"
36.     }
37. }
```

Tabel 6. Source Code MovieDatabase.kt

```
1.  package com.example.clickanime.data.local
2.
3.  import android.content.Context
4.  import androidx.room.Database
5.  import androidx.room.Room
6.  import androidx.room.RoomDatabase
7.  import
    com.example.clickanime.data.local.dao.FavoriteMovieD
    ao
```

```
8.  import com.example.clickanime.data.local.dao.MovieDao
9.  import
    com.example.clickanime.data.local.dao.MovieDetailDao
10. import
    com.example.clickanime.data.local.entity.FavoriteMov
    ieEntity
11. import
    com.example.clickanime.data.local.entity.MovieDetail
    Entity
12. import
    com.example.clickanime.data.local.entity.MovieEntity
13.
14. @Database(
15.     entities          =          [MovieEntity::class,
    MovieDetailEntity::class,
    FavoriteMovieEntity::class],
16.     version  = 1,
17.     exportSchema = false
18. )
19. abstract class MovieDatabase : RoomDatabase() {
20.
21.     abstract fun movieDao(): MovieDao
22.     abstract fun movieDetailDao(): MovieDetailDao
23.     abstract fun favoriteMovieDao(): FavoriteMovieDao
24.
25.     companion object {
26.         @Volatile     private     var     INSTANCE:
    MovieDatabase? = null
27.
28.         fun      getInstance(context:      Context):
    MovieDatabase =
29.             INSTANCE ?: synchronized(this) {
30.                 Room.databaseBuilder(
31.                     context.applicationContext,
32.                     MovieDatabase::class.java,
33.                     "clickanime_movie_db"
```

```
34.                 )
35.
.fallbackToDestructiveMigration()
36.                     .build()
37.                     .also { INSTANCE = it }
38.             }
39.     }
40. }
```

Tabel 7. Source Code MovieMapper.kt

```
1.  package com.example.clickanime.data.mapper
2.
3.  import
com.example.clickanime.data.local.entity.FavoriteMov
ieEntity
4.  import
com.example.clickanime.data.local.entity.MovieDetail
Entity
5.  import
com.example.clickanime.data.local.entity.MovieEntity
6.  import
com.example.clickanime.data.remote.model.TmdbGenreDt
o
7.  import
com.example.clickanime.data.remote.model.TmdbMovieDe
tailDto
8.  import
com.example.clickanime.data.remote.model.TmdbMovieDt
o
9.  import com.example.clickanime.model.FavoriteMovie
10. import com.example.clickanime.model.Genre
11. import com.example.clickanime.model.Movie
12. import com.example.clickanime.model.MovieDetail
13.
14. fun    TmdbMovieDto.toEntity(category:    String)    =
MovieEntity(
15.     id   =   id,   title   =   title,   originalTitle   =
originalTitle,
```

```
16.     overview = overview, posterPath = posterPath,
        backdropPath = backdropPath,
17.     voteAverage = voteAverage, voteCount = voteCount,
        releaseDate = releaseDate,
18.     genreIds = genreIds.joinToString(","), popularity
        = popularity,
19.     originalLanguage = originalLanguage, category =
        category,
20.     cachedAt = System.currentTimeMillis()
21. )
22.
23. fun          TmdbMovieDetailDto.toEntity()          =
        MovieDetailEntity(
24.     id   =   id,   title   =   title,   originalTitle   =
        originalTitle,
25.     overview = overview, posterPath = posterPath,
        backdropPath = backdropPath,
26.     voteAverage = voteAverage, voteCount = voteCount,
        releaseDate = releaseDate,
27.     runtime      =      runtime,      genres      =
        genres.joinToString(",") { "${it.id}:${it.name}" },
28.     tagline = tagline, status = status, popularity =
        popularity,
29.     budget = budget, revenue = revenue, homepage =
        homepage,
30.     originalLanguage = originalLanguage, cachedAt =
        System.currentTimeMillis()
31. )
32.
33. fun TmdbMovieDto.toDomain(category: String) = Movie(
34.     id   =   id,   title   =   title,   originalTitle   =
        originalTitle,
35.     overview = overview, posterPath = posterPath,
        backdropPath = backdropPath,
36.     voteAverage = voteAverage, voteCount = voteCount,
        releaseDate = releaseDate,
37.     genreIds = genreIds, popularity = popularity,
38.     originalLanguage = originalLanguage, category =
        category
```

```
)

fun TmdbMovieDetailDto.toDomain() = MovieDetail(
    id = id, title = title, originalTitle = originalTitle,
    overview = overview, posterPath = posterPath, backdropPath = backdropPath,
    voteAverage = voteAverage, voteCount = voteCount, releaseDate = releaseDate,
    runtime = runtime, genres = genres.map { Genre(it.id, it.name) },
    tagline = tagline, status = status, popularity = popularity,
    budget = budget, revenue = revenue, homepage = homepage,
    originalLanguage = originalLanguage
)

fun MovieEntity.toDomain() = Movie(
    id = id, title = title, originalTitle = originalTitle,
    overview = overview, posterPath = posterPath, backdropPath = backdropPath,
    voteAverage = voteAverage, voteCount = voteCount, releaseDate = releaseDate,
    genreIds = genreIds.split(",").mapNotNull { it.trim().toIntOrNull() },
    popularity = popularity, originalLanguage = originalLanguage, category = category
)

fun MovieDetailEntity.toDomain() = MovieDetail(
    id = id, title = title, originalTitle = originalTitle,
    overview = overview, posterPath = posterPath, backdropPath = backdropPath,
    voteAverage = voteAverage, voteCount = voteCount, releaseDate = releaseDate,
```

```
63.     runtime = runtime, genres = parseGenres(genres),
64.     tagline = tagline, status = status, popularity =
    popularity,
65.     budget = budget, revenue = revenue, homepage =
    homepage,
66.     originalLanguage = originalLanguage
67. )
68.
69. fun FavoriteMovieEntity.toDomain() = FavoriteMovie(
70.     movieId = movieId, title = title, posterPath =
    posterPath,
71.     voteAverage = voteAverage, releaseDate =
    releaseDate,
72.     overview = overview, addedAt = addedAt
73. )
74.
75. fun Movie.toFavoriteEntity() = FavoriteMovieEntity(
76.     movieId = id, title = title, posterPath =
    posterPath,
77.     voteAverage = voteAverage, releaseDate =
    releaseDate,
78.     overview = overview, addedAt =
    System.currentTimeMillis()
79. )
80.
81. fun MovieDetail.toFavoriteEntity() =
    FavoriteMovieEntity(
82.     movieId = id, title = title, posterPath =
    posterPath,
83.     voteAverage = voteAverage, releaseDate =
    releaseDate,
84.     overview = overview, addedAt =
    System.currentTimeMillis()
85. )
86.
87. private fun parseGenres(raw: String): List<Genre> {
88.     if (raw.isBlank()) return emptyList()
```

```
89.     return raw.split(",").mapNotNull { part ->
90.         val s = part.trim().split(":")
91.         if (s.size == 2) Genre(s[0].toIntOrNull() ?: return@mapNotNull null, s[1]) else null
92.     }
93. }
```

Tabel 8. Source Code TmdbResponse.kt

```
1.  package com.example.clickanime.data.remote.model
2.
3.  import kotlinx.serialization.SerialName
4.  import kotlinx.serialization.Serializable
5.
6.  sealed class ApiResponse<out T> {
7.      data class Success<T>(val data: T) : ApiResponse<T>()
8.      data class Error(val code: Int, val message: String) : ApiResponse<Nothing>()
9.      data class Exception(val e: Throwable) : ApiResponse<Nothing>()
10. }
11. @Serializable
12. data class TmdbMovieListResponse(
13.     @SerialName("page")          val page: Int = 1,
14.     @SerialName("results")              val results: List<TmdbMovieDto> = emptyList(),
15.     @SerialName("total_pages")    val totalPages: Int = 1,
16.     @SerialName("total_results")  val totalResults: Int = 0
17. )
18.
19. @Serializable
20. data class TmdbMovieDto(
21.     @SerialName("id")                val id: Int,
```

```
22.     @SerialName("title")            val title: String = "",
23.     @SerialName("original_title")               val originalTitle: String = "",
24.     @SerialName("overview")           val overview: String = "",
25.     @SerialName("poster_path")       val posterPath: String? = null,
26.     @SerialName("backdrop_path")    val backdropPath: String? = null,
27.     @SerialName("vote_average")      val voteAverage: Double = 0.0,
28.     @SerialName("vote_count")         val voteCount: Int = 0,
29.     @SerialName("release_date")      val releaseDate: String = "",
30.     @SerialName("genre_ids")           val genreIds: List<Int> = emptyList(),
31.     @SerialName("popularity")        val popularity: Double = 0.0,
32.     @SerialName("adult")           val adult: Boolean = false,
33.     @SerialName("original_language")             val originalLanguage: String = ""
34. )
35.
36. @Serializable
37. data class TmdbMovieDetailDto(
38.     @SerialName("id")               val id: Int,
39.     @SerialName("title")            val title: String = "",
40.     @SerialName("original_title")               val originalTitle: String = "",
41.     @SerialName("overview")           val overview: String = "",
42.     @SerialName("poster_path")       val posterPath: String? = null,
43.     @SerialName("backdrop_path")    val backdropPath: String? = null,
```

```
44.     @SerialName("vote_average")      val voteAverage:
        Double = 0.0,
45.     @SerialName("vote_count")          val voteCount:
        Int = 0,
46.     @SerialName("release_date")      val releaseDate:
        String = "",
47.     @SerialName("runtime")          val runtime: Int?
        = null,
48.     @SerialName("genres")                val genres:
        List<TmdbGenreDto> = emptyList(),
49.     @SerialName("tagline")              val tagline:
        String = "",
50.     @SerialName("status")          val status: String
        = "",
51.     @SerialName("popularity")         val popularity:
        Double = 0.0,
52.     @SerialName("budget")            val budget: Long
        = 0L,
53.     @SerialName("revenue")          val revenue: Long
        = 0L,
54.     @SerialName("homepage")             val homepage:
        String = "",
55.     @SerialName("original_language")              val
        originalLanguage: String = ""
56. )
57.
58. @Serializable
59. data class TmdbGenreDto(
60.     @SerialName("id")   val id: Int,
61.     @SerialName("name") val name: String
62. )
63.
64. @Serializable
65. data class TmdbGenreListResponse(
66.     @SerialName("genres")         val         genres:
        List<TmdbGenreDto> = emptyList()
67. )
```

Tabel 9. Source Code ApiHandler.kt

```
1.  package com.example.clickanime.data.remote
2.
3.  import com.example.clickanime.data.remote.model.ApiResponse
4.  import retrofit2.Response
5.  import timber.log.Timber
6.
7.  suspend fun <T> safeApiCall(apiCall: suspend () -> Response<T>): ApiResponse<T> {
8.      return try {
9.          val response = apiCall()
10.         if (response.isSuccessful) {
11.             val body = response.body()
12.             if (body != null) ApiResponse.Success(body)
13.             else ApiResponse.Error(response.code(), "Response body is null")
14.         } else {
15.             val errorMsg = response.errorBody()?.string() ?: "Unknown error"
16.             Timber.e("API Error [${response.code()}]: $errorMsg")
17.             ApiResponse.Error(response.code(), errorMsg)
18.         }
19.     } catch (e: Exception) {
20.         Timber.e(e, "API Exception")
21.         ApiResponse.Exception(e)
22.     }
23. }
```

Tabel 10. Source Code NetworkClient.kt

```
1.  package com.example.clickanime.data.remote
2.
```

```
3.  import com.example.clickanime.BuildConfig
4.  import com.jakewharton.retrofit2.converter.kotlinx.serialization.asConverterFactory
5.  import kotlinx.serialization.json.Json
6.  import okhttp3.MediaType.Companion.toMediaType
7.  import okhttp3.OkHttpClient
8.  import okhttp3.logging.HttpLoggingInterceptor
9.  import retrofit2.Retrofit
10. import timber.log.Timber
11. import java.util.concurrent.TimeUnit
12.
13. object NetworkClient {
14.
15.     private val json = Json {
16.         ignoreUnknownKeys = true
17.         coerceInputValues  = true
18.         encodeDefaults     = true
19.     }
20.
21.     private val authInterceptor = okhttp3.Interceptor { chain ->
22.         val req = chain.request().newBuilder()
23.             .addHeader("Authorization",      "Bearer ${BuildConfig.TMDB_API_KEY}")
24.             .addHeader("accept", "application/json")
25.             .build()
26.         chain.proceed(req)
27.     }
28.
29.     private      val      loggingInterceptor      = HttpLoggingInterceptor { msg ->
30.         Timber.tag("OkHttp").d(msg)
31.     }.apply {
```

```
32.         level = if (BuildConfig.DEBUG) HttpLoggingInterceptor.Level.BODY
33.         else HttpLoggingInterceptor.Level.NONE
34.     }
35.
36.     private val okHttpClient = OkHttpClient.Builder()
37.         .addInterceptor(authInterceptor)
38.         .addInterceptor(loggingInterceptor)
39.         .connectTimeout(30, TimeUnit.SECONDS)
40.         .readTimeout(30, TimeUnit.SECONDS)
41.         .writeTimeout(30, TimeUnit.SECONDS)
42.         .build()
43.
44.     private val retrofit = Retrofit.Builder()
45.         .baseUrl(BuildConfig.TMDB_BASE_URL)
46.         .client(okHttpClient)
47.         .addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
48.         .build()
49.
50.     val tmdbApiService: TmdbApiService = retrofit.create(TmdbApiService::class.java)
51. }
```

Tabel 11. Source Code TmdbApiService.kt

```
1.  package com.example.clickanime.data.remote
2.
3.  import com.example.clickanime.data.remote.model.TmdbGenreListResponse
4.  import com.example.clickanime.data.remote.model.TmdbMovieDetailDto
```

```
5.  import com.example.clickanime.data.remote.model.TmdbMovieListResponse
6.  import retrofit2.Response
7.  import retrofit2.http.GET
8.  import retrofit2.http.Path
9.  import retrofit2.http.Query
10.
11. interface TmdbApiService {
12.
13.     @GET("movie/now_playing")
14.     suspend fun getNowPlayingMovies(
15.         @Query("language") language: String = "en-US",
16.         @Query("page")     page: Int = 1
17.     ): Response<TmdbMovieListResponse>
18.
19.     @GET("movie/popular")
20.     suspend fun getPopularMovies(
21.         @Query("language") language: String = "en-US",
22.         @Query("page")     page: Int = 1
23.     ): Response<TmdbMovieListResponse>
24.
25.     @GET("movie/top_rated")
26.     suspend fun getTopRatedMovies(
27.         @Query("language") language: String = "en-US",
28.         @Query("page")     page: Int = 1
29.     ): Response<TmdbMovieListResponse>
30.
31.     @GET("movie/upcoming")
32.     suspend fun getUpcomingMovies(
33.         @Query("language") language: String = "en-US",
```

```
34.         @Query("page")     page: Int = 1
35.     ): Response<TmdbMovieListResponse>
36.
37.     @GET("movie/{movie_id}")
38.     suspend fun getMovieDetail(
39.         @Path("movie_id")  movieId: Int,
40.         @Query("language") language: String = "en-US"
41.     ): Response<TmdbMovieDetailDto>
42.
43.     @GET("search/movie")
44.     suspend fun searchMovies(
45.         @Query("query")    query: String,
46.         @Query("language")  language:  String  =  "en-
    US",
47.         @Query("page")     page: Int = 1
48.     ): Response<TmdbMovieListResponse>
49.
50.     @GET("genre/movie/list")
51.     suspend fun getMovieGenres(
52.         @Query("language") language: String = "en-US"
53.     ): Response<TmdbGenreListResponse>
54. }
```

Tabel 12. Source Code MovieRepository.kt

```
1.  package com.example.clickanime.data.repository
2.
3.  import
    com.example.clickanime.data.local.AppPreferences
4.  import
    com.example.clickanime.data.local.dao.FavoriteMovieD
    ao
5.  import
    com.example.clickanime.data.local.dao.MovieDao
```

```
6.  import com.example.clickanime.data.local.dao.MovieDetailDao
7.  import com.example.clickanime.data.mapper.toDomain
8.  import com.example.clickanime.data.mapper.toEntity
9.  import com.example.clickanime.data.mapper.toFavoriteEntity
10. import com.example.clickanime.data.remote.TmdbApiService
11. import com.example.clickanime.data.remote.model.ApiResponse
12. import com.example.clickanime.data.remote.safeApiCall
13. import com.example.clickanime.model.FavoriteMovie
14. import com.example.clickanime.model.Movie
15. import com.example.clickanime.model.MovieDetail
16. import kotlinx.coroutines.flow.Flow
17. import kotlinx.coroutines.flow.flow
18. import kotlinx.coroutines.flow.map
19. import timber.log.Timber
20.
21. class MovieRepository(
22.     private val api: TmdbApiService,
23.     private val movieDao: MovieDao,
24.     private val detailDao: MovieDetailDao,
25.     private val favDao: FavoriteMovieDao,
26.     private val prefs: AppPreferences
27. ) {
28.     companion object {
29.         private const val TTL_MS = 30 * 60 * 1000L
30.
31.         const val CAT_POPULAR     = "popular"
32.         const val CAT_NOW_PLAYING = "now_playing"
33.         const val CAT_TOP_RATED   = "top_rated"
34.         const val CAT_UPCOMING    = "upcoming"
```

```
35.     }
36.
37.     fun getMovies(
38.         category: String,
39.         forceRefresh: Boolean = false
40.     ): Flow<ApiResponse<List<Movie>>> = flow {
41.
42.         val      lastCache                          =
movieDao.getLatestCacheTime(category) ?: 0L
43.         val       cacheValid                        =
(System.currentTimeMillis() - lastCache) < TTL_MS
44.
45.         if (!cacheValid || forceRefresh) {
46.             Timber.d("Fetching  $category  from  API
(forceRefresh=$forceRefresh)")
47.             val result = fetchAndCache(category)
48.             if (result is ApiResponse.Error || result
is ApiResponse.Exception) {
49.                 if (lastCache == 0L) {
50.                     @Suppress("UNCHECKED_CAST")
51.                     emit(result                    as
ApiResponse<List<Movie>>)
52.                     return@flow
53.                 }
54.                 Timber.w("API  failed  for  $category,
using stale cache")
55.             }
56.         } else {
57.             Timber.d("Using    valid    cache    for
$category")
58.         }
59.
60.
movieDao.getMoviesByCategory(category).collect     {
entities ->
```

```
61.             emit(ApiResponse.Success(entities.map { it.toDomain() }))
62.         }
63.     }
64.
65.     private suspend fun fetchAndCache(category: String): ApiResponse<List<Movie>> {
66.         val response = safeApiCall {
67.             when (category) {
68.                 CAT_POPULAR -> api.getPopularMovies()
69.                 CAT_NOW_PLAYING -> api.getNowPlayingMovies()
70.                 CAT_TOP_RATED -> api.getTopRatedMovies()
71.                 CAT_UPCOMING -> api.getUpcomingMovies()
72.                 else -> api.getPopularMovies()
73.             }
74.         }
75.         return when (response) {
76.             is ApiResponse.Success -> {
77.                 val entities = response.data.results.map { it.toEntity(category) }
78.                 movieDao.deleteByCategory(category)
79.                 movieDao.insertMovies(entities)
80.                 Timber.d("Cached ${entities.size} movies [$category]")
81.                 ApiResponse.Success(entities.map { it.toDomain() })
82.             }
83.             else -> response as ApiResponse<List<Movie>>
84.         }
85.     }
86.
```

```
87.     suspend fun getMovieDetail(movieId: Int, forceRefresh: Boolean = false): ApiResponse<MovieDetail> {
88.         val lastCache = detailDao.getCacheTime(movieId) ?: 0L
89.         val cacheValid = (System.currentTimeMillis() - lastCache) < TTL_MS
90.
91.         if (cacheValid && !forceRefresh) {
92.             val cached = detailDao.getById(movieId)
93.             if (cached != null) {
94.                 Timber.d("Using cached detail for movieId=$movieId")
95.                 return ApiResponse.Success(cached.toDomain())
96.             }
97.         }
98.
99.         Timber.d("Fetching detail from API movieId=$movieId")
100.        return when (val r = safeApiCall { api.getMovieDetail(movieId) }) {
101.            is ApiResponse.Success -> {
102.                detailDao.insert(r.data.toEntity())
103.                ApiResponse.Success(r.data.toDomain())
104.            }
105.            else -> {
106.                // Fallback ke stale cache jika ada
107.                val stale = detailDao.getById(movieId)
108.                if (stale != null) ApiResponse.Success(stale.toDomain())
109.                else r as ApiResponse<MovieDetail>
110.            }
111.        }
112.    }
```

```
113.
114.     suspend fun searchMovies(query: String): ApiResponse<List<Movie>> {
115.         if (query.isBlank()) return ApiResponse.Success(emptyList())
116.         return when (val r = safeApiCall { api.searchMovies(query) }) {
117.             is ApiResponse.Success -> ApiResponse.Success(
118.                 r.data.results.map { it.toDomain(CAT_POPULAR) }
119.             )
120.             else -> r as ApiResponse<List<Movie>>
121.         }
122.     }
123.
124.     fun getAllFavorites(): Flow<List<FavoriteMovie>> =
125.         favDao.getAll().map { it.map { e -> e.toDomain() } }
126.
127.     fun isFavorite(movieId: Int): Flow<Boolean> = favDao.isFavorite(movieId)
128.
129.     fun getFavoriteCount(): Flow<Int> = favDao.count()
130.
131.     suspend fun addFavorite(movie: Movie) { favDao.add(movie.toFavoriteEntity()) }
132.     suspend fun addFavorite(detail: MovieDetail){ favDao.add(detail.toFavoriteEntity()) }
133.     suspend fun removeFavorite(movieId: Int) { favDao.removeById(movieId) }
134.
135.     suspend fun clearExpiredCache() {
136.         val exp = System.currentTimeMillis() - TTL_MS
137.         movieDao.deleteExpired(exp)
```

```
138.          detailDao.deleteExpired(exp)
139.          Timber.d("Expired cache cleared")
140.      }
141.
142.      suspend fun clearAllCache() {
143.          movieDao.clearAll()
144.          detailDao.clearAll()
145.          Timber.d("All cache cleared")
146.      }
147.
148.      var lastCategory: String
149.          get() = prefs.lastMovieCategory
150.          set(v) { prefs.lastMovieCategory = v }
151.  }
```

Tabel 13. Source Code AppContainer.kt

```
1.  package com.example.clickanime.di
2.
3.  import android.content.Context
4.  import
    com.example.clickanime.data.local.AppPreferences
5.  import
    com.example.clickanime.data.local.MovieDatabase
6.  import
    com.example.clickanime.data.remote.NetworkClient
7.  import
    com.example.clickanime.data.repository.MovieReposito
    ry
8.  import
    com.example.clickanime.viewmodel.MovieViewModelFacto
    ry
9.
10. class AppContainer(context: Context) {
11.
12.     val appPreferences = AppPreferences(context)
```

```
13.
14.     private            val            db            =
MovieDatabase.getInstance(context)
15.
16.     private val repo = MovieRepository(
17.         api        = NetworkClient.tmdbApiService,
18.         movieDao   = db.movieDao(),
19.         detailDao  = db.movieDetailDao(),
20.         favDao     = db.favoriteMovieDao(),
21.         prefs      = appPreferences
22.     )
23.
24.     val            movieViewModelFactory            =
MovieViewModelFactory(repo, appPreferences)
25. }
```

Tabel 14. Source Code Movie.kt

```
1.  package com.example.clickanime.model
2.
3.  private       const       val       IMG_BASE       =
    "https://image.tmdb.org/t/p/"
4.
5.  data class Movie(
6.      val id: Int,
7.      val title: String,
8.      val originalTitle: String,
9.      val overview: String,
10.     val posterPath: String?,
11.     val backdropPath: String?,
12.     val voteAverage: Double,
13.     val voteCount: Int,
14.     val releaseDate: String,
15.     val genreIds: List<Int>,
```

```
    val popularity: Double,
    val originalLanguage: String,
    val category: String
) {
    val posterUrl: String?   get() = posterPath?.let { "${IMG_BASE}w342$it" }
    val backdropUrl: String?   get() = backdropPath?.let { "${IMG_BASE}w780$it" }
    val releaseYear: String   get() = releaseDate.take(4).ifEmpty { "N/A" }
    val formattedRating: String   get() = String.format("%.1f", voteAverage)
}

data class MovieDetail(
    val id: Int,
    val title: String,
    val originalTitle: String,
    val overview: String,
    val posterPath: String?,
    val backdropPath: String?,
    val voteAverage: Double,
    val voteCount: Int,
    val releaseDate: String,
    val runtime: Int?,
    val genres: List<Genre>,
    val tagline: String,
    val status: String,
    val popularity: Double,
    val budget: Long,
    val revenue: Long,
    val homepage: String,
    val originalLanguage: String
) {
```

```
46.     val posterUrl: String?   get() = posterPath?.let { "${IMG_BASE}w500$it" }
47.     val backdropUrl: String? get() = backdropPath?.let { "${IMG_BASE}w780$it" }
48.     val releaseYear: String  get() = releaseDate.take(4).ifEmpty { "N/A" }
49.     val formattedRating: String  get() = String.format("%.1f", voteAverage)
50.     val formattedRuntime: String get() = runtime?.let { "${it / 60}h ${it % 60}m" } ?: "N/A"
51.     val genreNames: String get() = genres.joinToString(", ") { it.name }
52. }
53.
54. data class Genre(val id: Int, val name: String)
55.
56. data class FavoriteMovie(
57.     val movieId: Int,
58.     val title: String,
59.     val posterPath: String?,
60.     val voteAverage: Double,
61.     val releaseDate: String,
62.     val overview: String,
63.     val addedAt: Long
64. ) {
65.     val posterUrl: String?  get() = posterPath?.let { "${IMG_BASE}w342$it" }
66.     val releaseYear: String get() = releaseDate.take(4).ifEmpty { "N/A" }
67.     val formattedRating: String get() = String.format("%.1f", voteAverage)
68. }
```

Tabel 15. Source Code MovieUiState.kt

```
1. package com.example.clickanime.viewmodel
2.
```

```
3.  import com.example.clickanime.model.Movie
4.  import com.example.clickanime.model.MovieDetail
5.
6.  sealed class MovieListUiState {
7.      object Loading : MovieListUiState()
8.      data class Success(val movies: List<Movie>) : MovieListUiState()
9.      data class Error(val message: String) : MovieListUiState()
10. }
11.
12. sealed class MovieDetailUiState {
13.     object Idle : MovieDetailUiState()
14.     object Loading : MovieDetailUiState()
15.     data class Success(val movie: MovieDetail) : MovieDetailUiState()
16.     data class Error(val message: String) : MovieDetailUiState()
17. }
18.
19. sealed class MovieSearchUiState {
20.     object Idle : MovieSearchUiState()
21.     object Loading : MovieSearchUiState()
22.     data class Success(val movies: List<Movie>) : MovieSearchUiState()
23.     data class Error(val message: String) : MovieSearchUiState()
24.     object Empty : MovieSearchUiState()
25. }
```

Tabel 16. Source Code MovieViewModel.kt

```
1.  package com.example.clickanime.viewmodel
2.
3.  import androidx.lifecycle.ViewModel
```

```
4.  import androidx.lifecycle.ViewModelProvider
5.  import androidx.lifecycle.viewModelScope
6.  import com.example.clickanime.data.local.AppPreferences
7.  import com.example.clickanime.data.remote.model.ApiResponse
8.  import com.example.clickanime.data.repository.MovieRepository
9.  import com.example.clickanime.model.FavoriteMovie
10. import com.example.clickanime.model.Movie
11. import com.example.clickanime.model.MovieDetail
12. import kotlinx.coroutines.Job
13. import kotlinx.coroutines.delay
14. import kotlinx.coroutines.flow.*
15. import kotlinx.coroutines.launch
16. import timber.log.Timber
17.
18. class MovieViewModel(
19.     private val repo: MovieRepository,
20.     private val prefs: AppPreferences
21. ) : ViewModel() {
22.
23.     private val _selectedCategory = MutableStateFlow(
24.         prefs.lastMovieCategory.ifEmpty { MovieRepository.CAT_POPULAR }
25.     )
26.     val selectedCategory: StateFlow<String> = _selectedCategory.asStateFlow()
27.
28.     private val _listState = MutableStateFlow<MovieListUiState>(MovieListUiState.Loading)
29.     val listState: StateFlow<MovieListUiState> = _listState.asStateFlow()
30.
```

```
31.     private val _detailState = MutableStateFlow<MovieDetailUiState>(MovieDetailUiState.Idle)
32.     val detailState: StateFlow<MovieDetailUiState> = _detailState.asStateFlow()
33.
34.     private val _searchQuery = MutableStateFlow("")
35.     val searchQuery: StateFlow<String> = _searchQuery.asStateFlow()
36.
37.     private val _searchState = MutableStateFlow<MovieSearchUiState>(MovieSearchUiState.Idle)
38.     val searchState: StateFlow<MovieSearchUiState> = _searchState.asStateFlow()
39.
40.     val favorites: StateFlow<List<FavoriteMovie>> = repo.getAllFavorites()
41.         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), emptyList())
42.
43.     val favoriteCount: StateFlow<Int> = repo.getFavoriteCount()
44.         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), 0)
45.
46.     private var listJob: Job? = null
47.     private var searchJob: Job? = null
48.
49.     init {
50.         loadMovies(_selectedCategory.value)
51.         viewModelScope.launch { repo.clearExpiredCache() }
52.     }
53.
54.     fun selectCategory(cat: String) {
55.         if (_selectedCategory.value == cat) return
```

```
56.         _selectedCategory.value = cat
57.         prefs.lastMovieCategory = cat   // Simpan ke SharedPreferences
58.         loadMovies(cat)
59.         Timber.d("Category → $cat")
60.     }
61.
62.     fun loadMovies(category: String = _selectedCategory.value, forceRefresh: Boolean = false) {
63.         listJob?.cancel()
64.         listJob = viewModelScope.launch {
65.             _listState.value = MovieListUiState.Loading
66.             repo.getMovies(category, forceRefresh)
67.                 .catch { e ->
68.                     _listState.value = MovieListUiState.Error(e.message ?: "Unknown error")
69.                 }
70.                 .collect { response ->
71.                     _listState.value = when (response) {
72.                         is ApiResponse.Success -> {
73.                             if (response.data.isEmpty()) MovieListUiState.Loading
74.                             else MovieListUiState.Success(response.data)
75.                         }
76.                         is ApiResponse.Error -> MovieListUiState.Error("Error ${response.code}: ${response.message}")
77.                         is ApiResponse.Exception -> MovieListUiState.Error(response.e.message ?: "Network error")
78.                     }
79.                 }
80.         }
```

```
81.     }
82.
83.     fun refresh() = loadMovies(forceRefresh = true)
84.
85.     fun loadDetail(movieId: Int) {
86.         _detailState.value = MovieDetailUiState.Loading
87.         viewModelScope.launch {
88.             _detailState.value = when (val r = repo.getMovieDetail(movieId)) {
89.                 is ApiResponse.Success -> MovieDetailUiState.Success(r.data)
90.                 is ApiResponse.Error -> MovieDetailUiState.Error("Error ${r.code}: ${r.message}")
91.                 is ApiResponse.Exception -> MovieDetailUiState.Error(r.e.message ?: "Failed to load")
92.             }
93.         }
94.     }
95.
96.     fun clearDetail() { _detailState.value = MovieDetailUiState.Idle }
97.
98.     fun updateSearch(query: String) {
99.         _searchQuery.value = query
100.        searchJob?.cancel()
101.        if (query.isBlank()) { _searchState.value = MovieSearchUiState.Idle; return }
102.        searchJob = viewModelScope.launch {
103.            _searchState.value = MovieSearchUiState.Loading
104.            delay(500)   // debounce
105.            _searchState.value = when (val r = repo.searchMovies(query)) {
```

```
106.                 is ApiResponse.Success -> if (r.data.isEmpty()) MovieSearchUiState.Empty
107.                 else MovieSearchUiState.Success(r.data)
108.                 is ApiResponse.Error -> MovieSearchUiState.Error("Error ${r.code}: ${r.message}")
109.                 is ApiResponse.Exception -> MovieSearchUiState.Error(r.e.message ?: "Search failed")
110.             }
111.         }
112.     }
113.
114.     fun clearSearch() {
115.         _searchQuery.value = ""
116.         _searchState.value = MovieSearchUiState.Idle
117.         searchJob?.cancel()
118.     }
119.
120.     fun isFavoriteFlow(movieId: Int): Flow<Boolean> = repo.isFavorite(movieId)
121.
122.     fun toggleFavorite(movie: Movie) {
123.         viewModelScope.launch {
124.             val isFav = repo.isFavorite(movie.id).first()
125.             if (isFav) repo.removeFavorite(movie.id) else repo.addFavorite(movie)
126.         }
127.     }
128.
129.     fun toggleFavorite(detail: MovieDetail) {
130.         viewModelScope.launch {
131.             val isFav = repo.isFavorite(detail.id).first()
```

```
            if (isFav) repo.removeFavorite(detail.id) else repo.addFavorite(detail)
        }
    }

    fun removeFavorite(movieId: Int) {
        viewModelScope.launch { repo.removeFavorite(movieId) }
    }

    fun clearAllCache() {
        viewModelScope.launch {
            repo.clearAllCache()
            loadMovies(forceRefresh = true)
        }
    }
}

class MovieViewModelFactory(
    private val repo: MovieRepository,
    private val prefs: AppPreferences
) : ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        if (modelClass.isAssignableFrom(MovieViewModel::class.java)) {
            @Suppress("UNCHECKED_CAST")
            return MovieViewModel(repo, prefs) as T
        }
        throw IllegalArgumentException("Unknown ViewModel: ${modelClass.name}")
    }
}
```

Tabel 17. Source Code NavGraph.kt

```
1.  package com.example.clickanime.navigation
2.
3.  import androidx.compose.runtime.Composable
4.  import androidx.navigation.NavHostController
5.  import androidx.navigation.NavType
6.  import androidx.navigation.compose.NavHost
7.  import androidx.navigation.compose.composable
8.  import androidx.navigation.navArgument
9.  import
    com.example.clickanime.data.local.AppPreferences
10. import com.example.clickanime.ui.detail.DetailScreen
11. import
    com.example.clickanime.ui.favorites.FavoritesScreen
12. import com.example.clickanime.ui.home.HomeScreen
13. import
    com.example.clickanime.ui.language.LanguageScreen
14. import com.example.clickanime.ui.search.SearchScreen
15. import
    com.example.clickanime.ui.settings.SettingsScreen
16. import
    com.example.clickanime.viewmodel.MovieViewModel
17.
18. @Composable
19. fun NavGraph(
20.     navController: NavHostController,
21.     movieViewModel: MovieViewModel,
22.     appPreferences: AppPreferences,
23.     onLocaleChange: (String) -> Unit
24. ) {
25.     NavHost(
26.         navController  = navController,
27.         startDestination = Screen.Home.route
```

```
28.     ) {
29.         composable(Screen.Home.route) {
30.             HomeScreen(
31.                 viewModel      = movieViewModel,
32.                 onMovieClick          =    {    id    ->
    navController.navigate(Screen.Detail.createRoute(id))
    },
33.                 onSearchClick               =       {
    navController.navigate(Screen.Search.route) },
34.                 onFavoritesClick          =          {
    navController.navigate(Screen.Favorites.route) },
35.                 onLanguageClick          =          {
    navController.navigate(Screen.Language.route) },
36.                 onSettingsClick          =          {
    navController.navigate(Screen.Settings.route) }
37.             )
38.         }
39.
40.         composable(
41.             route     = Screen.Detail.route,
42.             arguments = listOf(navArgument("movieId")
    { type = NavType.IntType })
43.         ) { back ->
44.             val                movieId                =
    back.arguments?.getInt("movieId")                    ?:
    return@composable
45.             DetailScreen(
46.                 movieId   = movieId,
47.                 viewModel = movieViewModel,
48.                 onBack                          =      {
    navController.popBackStack() }
49.             )
50.         }
51.
52.         composable(Screen.Search.route) {
53.             SearchScreen(
```

```
54.                 viewModel    = movieViewModel,
55.                 onMovieClick     =     {     id     ->
navController.navigate(Screen.Detail.createRoute(id))
},
56.                 onBack                              =    {
navController.popBackStack() }
57.             )
58.         }
59.
60.         composable(Screen.Favorites.route) {
61.             FavoritesScreen(
62.                 viewModel    = movieViewModel,
63.                 onMovieClick     =     {     id     ->
navController.navigate(Screen.Detail.createRoute(id))
},
64.                 onBack                              =    {
navController.popBackStack() }
65.             )
66.         }
67.
68.         composable(Screen.Language.route) {
69.             LanguageScreen(
70.                 onLanguageSelected = { lang ->
71.                     onLocaleChange(lang)
72.                     navController.popBackStack()
73.                 },
74.                 onBack               =               {
navController.popBackStack() }
75.             )
76.         }
77.
78.         composable(Screen.Settings.route) {
79.             SettingsScreen(
80.                 viewModel    = movieViewModel,
81.                 preferences  = appPreferences,
```

```
                onBack                                =    {
navController.popBackStack() }
            )
        }
    }
}
```

Tabel 18. Source Code Screen.kt

```
package com.example.clickanime.navigation

sealed class Screen(val route: String) {
    object Home        : Screen("home")
    object Detail      : Screen("detail/{movieId}") {
        fun     createRoute(movieId:     Int)     =
"detail/$movieId"
    }
    object Search      : Screen("search")
    object Favorites   : Screen("favorites")
    object Settings    : Screen("settings")
    object Language    : Screen("language")
}
```

Tabel 19. Source Code HomeScreen.kt

```
package com.example.clickanime.ui.home

import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.LazyRow
import androidx.compose.foundation.lazy.items
import
androidx.compose.foundation.shape.RoundedCornerShape
```

```
10. import androidx.compose.material.icons.Icons
11. import androidx.compose.material.icons.filled.*
12. import androidx.compose.material3.*
13. import androidx.compose.runtime.*
14. import androidx.compose.ui.Alignment
15. import androidx.compose.ui.Modifier
16. import androidx.compose.ui.draw.clip
17. import androidx.compose.ui.graphics.Brush
18. import androidx.compose.ui.graphics.Color
19. import androidx.compose.ui.layout.ContentScale
20. import androidx.compose.ui.res.stringResource
21. import androidx.compose.ui.text.font.FontWeight
22. import androidx.compose.ui.text.style.TextOverflow
23. import androidx.compose.ui.unit.dp
24. import androidx.compose.ui.unit.sp
25. import coil.compose.AsyncImage
26. import com.example.clickanime.R
27. import
    com.example.clickanime.data.repository.MovieReposito
    ry
28. import com.example.clickanime.model.Movie
29. import
    com.example.clickanime.viewmodel.MovieListUiState
30. import
    com.example.clickanime.viewmodel.MovieViewModel
31.
32. private data class CategoryTab(val key: String, val
    labelRes: Int)
33.
34. private val CATEGORIES = listOf(
35.     CategoryTab(MovieRepository.CAT_POPULAR,
    R.string.category_popular),
36.     CategoryTab(MovieRepository.CAT_NOW_PLAYING,
    R.string.category_now_playing),
```

```
37.     CategoryTab(MovieRepository.CAT_TOP_RATED,
        R.string.category_top_rated),
38.     CategoryTab(MovieRepository.CAT_UPCOMING,
        R.string.category_upcoming),
39. )
40.
41. @OptIn(ExperimentalMaterial3Api::class)
42. @Composable
43. fun HomeScreen(
44.     viewModel: MovieViewModel,
45.     onMovieClick: (Int) -> Unit,
46.     onSearchClick: () -> Unit,
47.     onFavoritesClick: () -> Unit,
48.     onLanguageClick: () -> Unit,
49.     onSettingsClick: () -> Unit
50. ) {
51.     val    listState                                    by
        viewModel.listState.collectAsState()
52.     val              selectedCategory                   by
        viewModel.selectedCategory.collectAsState()
53.     val    favCount                                     by
        viewModel.favoriteCount.collectAsState()
54.
55.     Scaffold(
56.         topBar = {
57.             TopAppBar(
58.                 title               =                  {
        Text(stringResource(R.string.app_title), fontWeight =
        FontWeight.Bold) },
59.                 actions = {
60.                     IconButton(onClick                 =
        onSearchClick) {
61.                         Icon(Icons.Default.Search,
        contentDescription                                 =
        stringResource(R.string.nav_search))
62.                     }
```

```
63.                     BadgedBox(badge = {
64.                         if (favCount > 0) Badge { Text(favCount.toString()) }
65.                     }) {
66.                         IconButton(onClick = onFavoritesClick) {
67.                             Icon(Icons.Default.Favorite, contentDescription = stringResource(R.string.nav_favorites))
68.                         }
69.                     }
70.                     IconButton(onClick = onLanguageClick) {
71.                         Icon(Icons.Default.Language, contentDescription = stringResource(R.string.change_language))
72.                     }
73.                     IconButton(onClick = onSettingsClick) {
74.                         Icon(Icons.Default.Settings, contentDescription = stringResource(R.string.nav_settings))
75.                     }
76.                 },
77.                 colors = TopAppBarDefaults.topAppBarColors(
78.                     containerColor = MaterialTheme.colorScheme.primary,
79.                     titleContentColor = MaterialTheme.colorScheme.onPrimary,
80.                     actionIconContentColor = MaterialTheme.colorScheme.onPrimary
81.                 )
82.             )
83.         }
84.     ) { padding ->
85.         Column(
86.             modifier = Modifier
```

```
87.                 .fillMaxSize()
88.                 .padding(padding)
89.         ) {
90.             val selectedIdx = CATEGORIES.indexOfFirst { it.key == selectedCategory }.coerceAtLeast(0)
91.             ScrollableTabRow(
92.                 selectedTabIndex = selectedIdx,
93.                 edgePadding      = 8.dp,
94.                 containerColor                          = MaterialTheme.colorScheme.surface,
95.                 contentColor                            = MaterialTheme.colorScheme.primary
96.             ) {
97.                 CATEGORIES.forEach { cat ->
98.                     val selected = selectedCategory == cat.key
99.                     Tab(
100.                        selected = selected,
101.                        onClick             =       { viewModel.selectCategory(cat.key) },
102.                        text     = {
103.                            Text(
104.                                text                = stringResource(cat.labelRes),
105.                                fontWeight    =    if (selected) FontWeight.Bold else FontWeight.Normal
106.                            )
107.                        }
108.                    )
109.                }
110.            }
111.
112.            when (val state = listState) {
113.                is    MovieListUiState.Loading    -> LoadingContent()
```

```
                is  MovieListUiState.Error       -> ErrorContent(state.message) { viewModel.refresh() }
                is    MovieListUiState.Success    -> MovieContent(
                    movies      = state.movies,
                    viewModel   = viewModel,
                    onMovieClick = onMovieClick
                )
            }
        }
    }
}

@Composable
private fun MovieContent(
    movies: List<Movie>,
    viewModel: MovieViewModel,
    onMovieClick: (Int) -> Unit
) {
    LazyColumn(
        contentPadding   =   PaddingValues(bottom   = 16.dp),
        modifier       = Modifier.fillMaxSize()
    ) {
        if (movies.isNotEmpty()) {
            item {
                Text(
                    text                              = stringResource(R.string.featured),
                    style                             = MaterialTheme.typography.titleMedium,
                    fontWeight = FontWeight.Bold,
                    modifier = Modifier.padding(start = 16.dp, top = 12.dp, bottom = 8.dp)
                )
```

```
143.                 MovieCarousel(movies = movies.take(6), onMovieClick = onMovieClick)
144.                 Spacer(Modifier.height(12.dp))
145.                 HorizontalDivider(modifier = Modifier.padding(horizontal = 16.dp))
146.                 Text(
147.                     text = "${stringResource(R.string.all_movies)} (${movies.size})",
148.                     style = MaterialTheme.typography.titleMedium,
149.                     fontWeight = FontWeight.Bold,
150.                     modifier = Modifier.padding(start = 16.dp, top = 10.dp, bottom = 4.dp)
151.                 )
152.             }
153.         }
154.
155.         items(movies, key = { it.id }) { movie ->
156.             MovieListItem(
157.                 movie = movie,
158.                 onMovieClick = { onMovieClick(movie.id) },
159.                 onFavoriteClick = { viewModel.toggleFavorite(movie) },
160.                 isFavoriteFlow = viewModel.isFavoriteFlow(movie.id)
161.             )
162.         }
163.     }
164. }
165.
166. @Composable
167. private fun MovieCarousel(movies: List<Movie>, onMovieClick: (Int) -> Unit) {
168.     LazyRow(
```

```
        contentPadding     = PaddingValues(horizontal = 16.dp),
        horizontalArrangement                          = Arrangement.spacedBy(12.dp)
    ) {
        items(movies, key = { it.id }) { movie ->
            CarouselCard(movie = movie, onClick = { onMovieClick(movie.id) })
        }
    }
}

@Composable
private fun CarouselCard(movie: Movie, onClick: () -> Unit) {
    Card(
        modifier  = Modifier
            .width(160.dp)
            .height(220.dp)
            .clickable { onClick() },
        shape     = RoundedCornerShape(16.dp),
        elevation = CardDefaults.cardElevation(6.dp)
    ) {
        Box(Modifier.fillMaxSize()) {
            AsyncImage(
                model              = movie.posterUrl,
                contentDescription = movie.title,
                contentScale     = ContentScale.Crop,
                modifier                                = Modifier.fillMaxSize()
            )
            Box(
                Modifier.fillMaxSize().background(
                    Brush.verticalGradient(
```

```
                        colors                        = listOf(Color.Transparent, Color.Black.copy(.80f)),
                        startY  = 120f
                    )
                )
            )
            Column(
                Modifier.align(Alignment.BottomStart).padding(10.dp)
            ) {
                Text(movie.title,       color       = Color.White, fontWeight = FontWeight.Bold,
                    fontSize = 12.sp, maxLines = 2, overflow = TextOverflow.Ellipsis)
                Row(verticalAlignment               = Alignment.CenterVertically) {
                    Icon(Icons.Default.Star,    null, tint     =     Color(0xFFFFD700),     modifier     = Modifier.size(12.dp))
                    Spacer(Modifier.width(3.dp))
                    Text(movie.formattedRating, color = Color.White, fontSize = 11.sp)
                    Spacer(Modifier.width(6.dp))
                    Text(movie.releaseYear,  color  = Color.White.copy(.7f), fontSize = 11.sp)
                }
            }
        }
    }
}

@Composable
private fun MovieListItem(
    movie: Movie,
    onMovieClick: () -> Unit,
    onFavoriteClick: () -> Unit,
```

```
225.     isFavoriteFlow:
kotlinx.coroutines.flow.Flow<Boolean>
226. ) {
227.     val                    isFav                    by
isFavoriteFlow.collectAsState(initial = false)
228.
229.     Card(
230.         modifier  = Modifier
231.             .fillMaxWidth()
232.             .padding(horizontal = 16.dp, vertical =
5.dp)
233.             .clickable { onMovieClick() },
234.         shape     = RoundedCornerShape(14.dp),
235.         elevation = CardDefaults.cardElevation(3.dp),
236.         colors                                        =
CardDefaults.cardColors(containerColor                =
MaterialTheme.colorScheme.surface)
237.     ) {
238.         Row(
239.             modifier                                  =
Modifier.fillMaxWidth().padding(10.dp),
240.             verticalAlignment = Alignment.Top
241.         ) {
242.             AsyncImage(
243.                 model              = movie.posterUrl,
244.                 contentDescription = movie.title,
245.                 contentScale                          =
ContentScale.Crop,
246.                 modifier           = Modifier
247.                     .width(72.dp).height(105.dp)
248.                     .clip(RoundedCornerShape(10.dp))
249.             )
250.             Spacer(Modifier.width(12.dp))
251.             Column(Modifier.weight(1f)) {
252.                 Row(
```

```
253.                     Modifier.fillMaxWidth(),
254.                     horizontalArrangement           =
    Arrangement.SpaceBetween,
255.                     verticalAlignment               =
    Alignment.CenterVertically
256.                 ) {
257.                     Text(
258.                         movie.title,
259.                         style                       =
    MaterialTheme.typography.titleSmall,
260.                         fontWeight = FontWeight.Bold,
261.                         maxLines   = 2,
262.                         overflow                    =
    TextOverflow.Ellipsis,
263.                         modifier                    =
    Modifier.weight(1f)
264.                     )
265.                     IconButton(onClick              =
    onFavoriteClick, modifier = Modifier.size(36.dp)) {
266.                         Icon(
267.                             imageVector          = if
    (isFav)          Icons.Default.Favorite          else
    Icons.Default.FavoriteBorder,
268.                             contentDescription  =  if
    (isFav)
    stringResource(R.string.remove_from_favorites)
269.                             else
    stringResource(R.string.add_to_favorites),
270.                             tint                = if
    (isFav)                 Color.Red                else
    MaterialTheme.colorScheme.onSurfaceVariant,
271.                             modifier                =
    Modifier.size(20.dp)
272.                         )
273.                     }
274.                 }
275.                 Spacer(Modifier.height(3.dp))
```

```
276.                 Row(verticalAlignment = Alignment.CenterVertically) {
277.                     Icon(Icons.Default.Star, null, tint = Color(0xFFFFD700), modifier = Modifier.size(14.dp))
278.                     Spacer(Modifier.width(3.dp))
279.                     Text("${movie.formattedRating} (${movie.voteCount})",
280.                         style = MaterialTheme.typography.bodySmall,
281.                         color = MaterialTheme.colorScheme.onSurfaceVariant)
282.                     Spacer(Modifier.width(8.dp))
283.                     Icon(Icons.Default.CalendarToday, null,
284.                         tint = MaterialTheme.colorScheme.primary, modifier = Modifier.size(12.dp))
285.                     Spacer(Modifier.width(2.dp))
286.                     Text(movie.releaseYear, style = MaterialTheme.typography.bodySmall,
287.                         color = MaterialTheme.colorScheme.primary)
288.                 }
289.                 Spacer(Modifier.height(5.dp))
290.                 Text(
291.                     movie.overview,
292.                     style = MaterialTheme.typography.bodySmall,
293.                     color = MaterialTheme.colorScheme.onSurfaceVariant,
294.                     maxLines = 3, overflow = TextOverflow.Ellipsis,
295.                     lineHeight = 18.sp
296.                 )
297.             }
298.         }
299.     }
```

```
300. }
301.
302. @Composable
303. private fun LoadingContent() {
304.     Box(Modifier.fillMaxSize(),  contentAlignment  =
     Alignment.Center) {
305.         Column(horizontalAlignment                  =
     Alignment.CenterHorizontally) {
306.             CircularProgressIndicator(color         =
     MaterialTheme.colorScheme.primary)
307.             Spacer(Modifier.height(16.dp))
308.
     Text(stringResource(R.string.loading_movies), color =
     MaterialTheme.colorScheme.onSurfaceVariant)
309.         }
310.     }
311. }
312.
313. @Composable
314. private fun ErrorContent(message: String, onRetry: ()
     -> Unit) {
315.     Box(Modifier.fillMaxSize(),  contentAlignment  =
     Alignment.Center) {
316.         Column(horizontalAlignment                  =
     Alignment.CenterHorizontally,       modifier       =
     Modifier.padding(32.dp)) {
317.             Icon(Icons.Default.ErrorOutline, null,
318.                 tint                                =
     MaterialTheme.colorScheme.error,     modifier      =
     Modifier.size(64.dp))
319.             Spacer(Modifier.height(16.dp))
320.
     Text(stringResource(R.string.failed_to_load), style =
     MaterialTheme.typography.titleMedium,
321.                 fontWeight = FontWeight.Bold)
322.             Spacer(Modifier.height(8.dp))
323.             Text(message,           style           =
     MaterialTheme.typography.bodySmall,
```

```
                color                                   =
MaterialTheme.colorScheme.onSurfaceVariant)
            Spacer(Modifier.height(20.dp))
            Button(onClick = onRetry) {
                Icon(Icons.Default.Refresh, null)
                Spacer(Modifier.width(8.dp))
                Text(stringResource(R.string.retry))
            }
        }
    }
}
```

Tabel 20. Source Code DetailScreen.kt

```
package com.example.clickanime.ui.detail

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import
androidx.compose.foundation.rememberScrollState
import
androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import
androidx.compose.material.icons.automirrored.filled.A
rrowBack
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
```

```
18. import androidx.compose.ui.graphics.vector.ImageVector
19. import androidx.compose.ui.layout.ContentScale
20. import androidx.compose.ui.res.stringResource
21. import androidx.compose.ui.text.font.FontWeight
22. import androidx.compose.ui.text.style.TextAlign
23. import androidx.compose.ui.unit.dp
24. import androidx.compose.ui.unit.sp
25. import coil.compose.AsyncImage
26. import com.example.clickanime.R
27. import com.example.clickanime.model.MovieDetail
28. import com.example.clickanime.viewmodel.MovieDetailUiState
29. import com.example.clickanime.viewmodel.MovieViewModel
30.
31. @OptIn(ExperimentalMaterial3Api::class)
32. @Composable
33. fun DetailScreen(
34.     movieId: Int,
35.     viewModel: MovieViewModel,
36.     onBack: () -> Unit
37. ) {
38.     val uiState by viewModel.detailState.collectAsState()
39.
40.     LaunchedEffect(movieId) { viewModel.loadDetail(movieId) }
41.     DisposableEffect(Unit) { onDispose { viewModel.clearDetail() } }
42.
43.     Scaffold(
44.         topBar = {
45.             TopAppBar(
46.                 title = {
```

```
47.                     val title = (uiState as? MovieDetailUiState.Success)?.movie?.title ?: ""
48.                     Text(title, fontWeight = FontWeight.Bold, maxLines = 1, overflow = androidx.compose.ui.text.style.TextOverflow.Ellipsis)
49.                 },
50.                 navigationIcon = {
51.                     IconButton(onClick = onBack) {
52.                         Icon(Icons.AutoMirrored.Filled.ArrowBack, stringResource(R.string.back))
53.                     }
54.                 },
55.                 actions = {
56.                     if (uiState is MovieDetailUiState.Success) {
57.                         val movie = (uiState as MovieDetailUiState.Success).movie
58.                         val isFav by viewModel.isFavoriteFlow(movie.id).collectAsState(false)
59.                         IconButton(onClick = { viewModel.toggleFavorite(movie) }) {
60.                             Icon(
61.                                 imageVector = if (isFav) Icons.Default.Favorite else Icons.Default.FavoriteBorder,
62.                                 contentDescription = if (isFav) stringResource(R.string.remove_from_favorites)
63.                                 else stringResource(R.string.add_to_favorites),
64.                                 tint = if (isFav) Color.Red else MaterialTheme.colorScheme.onPrimary
65.                             )
66.                         }
67.                     }
68.                 },
```

```
69.                 colors                                  =
    TopAppBarDefaults.topAppBarColors(
70.                     containerColor                      =
    MaterialTheme.colorScheme.primary,
71.                     titleContentColor                   =
    MaterialTheme.colorScheme.onPrimary,
72.                     navigationIconContentColor          =
    MaterialTheme.colorScheme.onPrimary
73.                 )
74.             )
75.         }
76.     ) { padding ->
77.         when (val state = uiState) {
78.             is            MovieDetailUiState.Loading,
    MovieDetailUiState.Idle -> {
79.
    Box(Modifier.fillMaxSize().padding(padding),
    contentAlignment = Alignment.Center) {
80.                     CircularProgressIndicator()
81.                 }
82.             }
83.             is MovieDetailUiState.Error -> {
84.
    Box(Modifier.fillMaxSize().padding(padding),
    contentAlignment = Alignment.Center) {
85.                     Column(horizontalAlignment          =
    Alignment.CenterHorizontally,       modifier        =
    Modifier.padding(24.dp)) {
86.
    Icon(Icons.Default.ErrorOutline,    null,    tint    =
    MaterialTheme.colorScheme.error,      modifier      =
    Modifier.size(56.dp))
87.
    Spacer(Modifier.height(12.dp))
88.                         Text(state.message, textAlign
    = TextAlign.Center)
89.
    Spacer(Modifier.height(16.dp))
```

```
90.                         Button(onClick = { viewModel.loadDetail(movieId) }) { Text(stringResource(R.string.retry)) }
91.                     }
92.                 }
93.             }
94.             is MovieDetailUiState.Success -> {
95.                 DetailContent(movie = state.movie, paddingValues = padding)
96.             }
97.         }
98.     }
99. }
100.
101. @Composable
102. private fun DetailContent(movie: MovieDetail, paddingValues: PaddingValues) {
103.     Column(
104.         modifier = Modifier
105.             .fillMaxSize()
106.             .padding(paddingValues)
107.             .verticalScroll(rememberScrollState())
108.     ) {
109.         Box(Modifier.fillMaxWidth().height(260.dp)) {
110.             AsyncImage(
111.                 model = movie.backdropUrl ?: movie.posterUrl,
112.                 contentDescription = movie.title,
113.                 contentScale = ContentScale.Crop,
114.                 modifier = Modifier.fillMaxSize()
115.             )
116.             Box(Modifier.fillMaxSize().background(
```

```
117.
Brush.verticalGradient(listOf(Color.Transparent,
MaterialTheme.colorScheme.background))
118.            ))
119.
Column(Modifier.align(Alignment.BottomStart).padding(
16.dp)) {
120.                Text(movie.title,       color       =
MaterialTheme.colorScheme.onBackground,
121.                    fontWeight                      =
FontWeight.ExtraBold, fontSize = 22.sp)
122.                if (movie.tagline.isNotBlank()) {
123.                    Text("\"${movie.tagline}\"",
color = MaterialTheme.colorScheme.primary,
124.                        fontSize = 13.sp, modifier =
Modifier.padding(top = 2.dp))
125.                }
126.            }
127.        }
128.
129.
Row(Modifier.fillMaxWidth().padding(horizontal      =
16.dp).offset(y = (-20).dp)) {
130.            AsyncImage(
131.                model              = movie.posterUrl,
132.                contentDescription = movie.title,
133.                contentScale     = ContentScale.Crop,
134.                modifier                            =
Modifier.width(100.dp).height(148.dp).clip(RoundedCor
nerShape(12.dp))
135.            )
136.            Spacer(Modifier.width(14.dp))
137.            Card(
138.                modifier                            =
Modifier.weight(1f).align(Alignment.Bottom),
139.                shape    = RoundedCornerShape(12.dp),
```

```
                colors                                  =
CardDefaults.cardColors(containerColor                  =
MaterialTheme.colorScheme.primaryContainer)
            ) {
                Column(Modifier.padding(14.dp),
verticalArrangement = Arrangement.spacedBy(8.dp)) {
                    StatRow(Icons.Default.Star,
stringResource(R.string.rating),
"${movie.formattedRating}/10", Color(0xFFFFD700))
                    StatRow(Icons.Default.Schedule,
stringResource(R.string.runtime),
movie.formattedRuntime,
MaterialTheme.colorScheme.primary)

StatRow(Icons.Default.CalendarToday,
stringResource(R.string.year),     movie.releaseYear,
MaterialTheme.colorScheme.secondary)
                    StatRow(Icons.Default.HowToVote,
stringResource(R.string.votes),
movie.voteCount.toString(),
MaterialTheme.colorScheme.tertiary)
                }
            }
        }


Column(Modifier.fillMaxWidth().padding(horizontal   =
16.dp)) {
            if (movie.genres.isNotEmpty()) {
                Row(horizontalArrangement           =
Arrangement.spacedBy(8.dp),         modifier        =
Modifier.padding(bottom = 10.dp)) {
                    movie.genres.take(4).forEach    {
genre ->
                        Surface(shape               =
RoundedCornerShape(20.dp),          color           =
MaterialTheme.colorScheme.secondaryContainer) {
                            Text(genre.name, modifier
= Modifier.padding(horizontal = 12.dp, vertical =
5.dp),
                                style               =
MaterialTheme.typography.labelSmall,
```

```
158.                                    color                =
    MaterialTheme.colorScheme.onSecondaryContainer,
159.                                    fontWeight           =
    FontWeight.SemiBold)
160.                            }
161.                        }
162.                    }
163.                }
164.
165.                if (movie.status.isNotBlank()) {
166.                    Row(verticalAlignment                =
    Alignment.CenterVertically,         modifier             =
    Modifier.padding(bottom = 12.dp)) {
167.                        Icon(Icons.Default.Info,     null,
    Modifier.size(14.dp),               tint                 =
    MaterialTheme.colorScheme.primary)
168.                        Spacer(Modifier.width(4.dp))
169.
    Text("${stringResource(R.string.status)}:
    ${movie.status}",
170.                            style                        =
    MaterialTheme.typography.bodySmall,      color           =
    MaterialTheme.colorScheme.primary)
171.                        Spacer(Modifier.width(12.dp))
172.
    Text("${stringResource(R.string.language)}:
    ${movie.originalLanguage.uppercase()}",
173.                            style                        =
    MaterialTheme.typography.bodySmall,      color           =
    MaterialTheme.colorScheme.onSurfaceVariant)
174.                    }
175.                }
176.
177.                Text(stringResource(R.string.overview),
    style = MaterialTheme.typography.titleMedium,
178.                    fontWeight     =     FontWeight.Bold,
    modifier = Modifier.padding(bottom = 8.dp))
179.                Text(
```

```
180.                 text       = movie.overview.ifBlank { stringResource(R.string.no_overview) },
181.                 style                                = MaterialTheme.typography.bodyMedium,
182.                 color                                = MaterialTheme.colorScheme.onSurfaceVariant,
183.                 lineHeight = 22.sp,
184.                 modifier   = Modifier.padding(bottom = 20.dp)
185.             )
186.
187.             if (movie.budget > 0 || movie.revenue > 0) {
188.
Text(stringResource(R.string.box_office),   style   = MaterialTheme.typography.titleMedium,
189.                     fontWeight   =   FontWeight.Bold, modifier = Modifier.padding(bottom = 8.dp))
190.                 Card(
191.                     modifier                             = Modifier.fillMaxWidth().padding(bottom = 20.dp),
192.                     shape                                = RoundedCornerShape(12.dp),
193.                     colors                               = CardDefaults.cardColors(containerColor             = MaterialTheme.colorScheme.surfaceVariant)
194.                 ) {
195.
Row(Modifier.fillMaxWidth().padding(16.dp), horizontalArrangement = Arrangement.SpaceEvenly) {
196.
BoxOfficeCol(stringResource(R.string.budget), formatCurrency(movie.budget))
197.
VerticalDivider(Modifier.height(40.dp))
198.
BoxOfficeCol(stringResource(R.string.revenue), formatCurrency(movie.revenue))
199.                     }
200.                 }
```

```
            }

            Spacer(Modifier.height(24.dp))
        }
    }
}

@Composable
private fun StatRow(icon: ImageVector, label: String, value: String, tint: Color) {
    Row(verticalAlignment = Alignment.CenterVertically) {
        Icon(icon, null, tint = tint, modifier = Modifier.size(16.dp))
        Spacer(Modifier.width(6.dp))
        Column {
            Text(label, style = MaterialTheme.typography.labelSmall, color = MaterialTheme.colorScheme.onSurfaceVariant)
            Text(value, style = MaterialTheme.typography.bodySmall, fontWeight = FontWeight.SemiBold)
        }
    }
}

@Composable
private fun BoxOfficeCol(label: String, value: String) {
    Column(horizontalAlignment = Alignment.CenterHorizontally) {
        Text(label, style = MaterialTheme.typography.labelMedium, color = MaterialTheme.colorScheme.onSurfaceVariant)
        Text(value, fontWeight = FontWeight.Bold)
    }
}
```

```
227.
228. private fun formatCurrency(amount: Long): String = when {
229.     amount == 0L            -> "N/A"
230.     amount >= 1_000_000_000 -> "$%.1fB".format(amount / 1_000_000_000.0)
231.     amount >= 1_000_000     -> "$%.1fM".format(amount / 1_000_000.0)
232.     else                    -> "$$amount"
233. }
```

Tabel 21. Source Code LanguageScreen.kt

```
1.  package com.example.clickanime.ui.language
2.
3.  import androidx.compose.foundation.border
4.  import androidx.compose.foundation.clickable
5.  import androidx.compose.foundation.layout.*
6.  import androidx.compose.foundation.shape.RoundedCornerShape
7.  import androidx.compose.material.icons.Icons
8.  import androidx.compose.material.icons.automirrored.filled.ArrowBack
9.  import androidx.compose.material.icons.filled.Check
10. import androidx.compose.material.icons.filled.Language
11. import androidx.compose.material3.*
12. import androidx.compose.runtime.*
13. import androidx.compose.ui.Alignment
14. import androidx.compose.ui.Modifier
15. import androidx.compose.ui.platform.LocalContext
16. import androidx.compose.ui.res.stringResource
17. import androidx.compose.ui.text.font.FontWeight
18. import androidx.compose.ui.unit.dp
19. import androidx.compose.ui.unit.sp
```

```
import com.example.clickanime.R

private data class LangOption(
    val code: String,
    val nativeName: String,
    val labelRes: Int,
    val flag: String
)

private val LANGUAGES = listOf(
    LangOption("en",                          "English",
R.string.lang_english,    "GB"),
    LangOption("id",        "Bahasa       Indonesia",
R.string.lang_indonesian, "ID"),
)

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun LanguageScreen(
    onLanguageSelected: (String) -> Unit,
    onBack: () -> Unit
) {
    val context = LocalContext.current
    val                 currentLang                 =
context.resources.configuration.locales[0].language

    var       selected       by       remember       {
mutableStateOf(currentLang) }

    Scaffold(
        topBar = {
            TopAppBar(
                title = {
                    Text(
```

```
                        text                            = stringResource(R.string.change_language),
                        fontWeight = FontWeight.Bold
                    )
                },
                navigationIcon = {
                    IconButton(onClick = onBack) {
                        Icon(
                            imageVector             = Icons.AutoMirrored.Filled.ArrowBack,
                            contentDescription      = stringResource(R.string.back)
                        )
                    }
                },
                colors                              = TopAppBarDefaults.topAppBarColors(
                    containerColor                  = MaterialTheme.colorScheme.primary,
                    titleContentColor               = MaterialTheme.colorScheme.onPrimary,
                    navigationIconContentColor      = MaterialTheme.colorScheme.onPrimary
                )
            )
        }
    ) { padding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding)
                .padding(24.dp)
        ) {
            Icon(
                imageVector = Icons.Default.Language,
```

```
                contentDescription = null,
                modifier = Modifier
                    .size(64.dp)
.align(Alignment.CenterHorizontally),
                tint                                = MaterialTheme.colorScheme.primary
            )
            Spacer(Modifier.height(8.dp))
            Text(
                text                                = stringResource(R.string.select_language),
                style                               = MaterialTheme.typography.titleLarge,
                fontWeight = FontWeight.Bold,
                modifier                            = Modifier.align(Alignment.CenterHorizontally)
            )
            Spacer(Modifier.height(28.dp))

            LANGUAGES.forEach { lang ->
                val isSelected = selected == lang.code
                Card(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(vertical = 6.dp)
                        .then(
                            if           (isSelected) Modifier.border(
                                width = 2.dp,
                                color               = MaterialTheme.colorScheme.primary,
                                shape               = RoundedCornerShape(14.dp)
                            ) else Modifier
```

```
105.                            )
106.                            .clickable { selected = lang.code },
107.                        shape                               = RoundedCornerShape(14.dp),
108.                        colors = CardDefaults.cardColors(
109.                            containerColor      =      if (isSelected)
110.                                MaterialTheme.colorScheme.primaryContainer
111.                            else
112.                                MaterialTheme.colorScheme.surface
113.                        ),
114.                        elevation                           = CardDefaults.cardElevation(
115.                            defaultElevation     =     if (isSelected) 4.dp else 1.dp
116.                        )
117.                    ) {
118.                        Row(
119.                            modifier = Modifier
120.                                .fillMaxWidth()
121.                                .padding(18.dp),
122.                            verticalAlignment           = Alignment.CenterVertically
123.                        ) {
124.                            Text(
125.                                text = lang.flag,
126.                                fontSize = 32.sp,
127.                                modifier                = Modifier.padding(end = 16.dp)
128.                            )
129.                            Column(Modifier.weight(1f)) {
130.                                Text(
```

```
131.                                     text                  =
    lang.nativeName,
132.                                     fontWeight            =
    FontWeight.Bold,
133.                                     style                 =
    MaterialTheme.typography.bodyLarge
134.                                 )
135.                                 Text(
136.                                     text                  =
    stringResource(lang.labelRes),
137.                                     style                 =
    MaterialTheme.typography.bodySmall,
138.                                     color                 =
    MaterialTheme.colorScheme.onSurfaceVariant
139.                                 )
140.                             }
141.                             if (isSelected) {
142.                                 Icon(
143.                                     imageVector           =
    Icons.Default.Check,
144.                                     contentDescription    =
    null,
145.                                     tint                  =
    MaterialTheme.colorScheme.primary,
146.                                     modifier              =
    Modifier.size(24.dp)
147.                                 )
148.                             }
149.                         }
150.                     }
151.                 }
152.
153.                 Spacer(Modifier.weight(1f))
154.
155.                 Button(
156.                     onClick               =               {
    onLanguageSelected(selected) },
```

```
157.                 modifier = Modifier
158.                     .fillMaxWidth()
159.                     .height(52.dp),
160.                 shape = RoundedCornerShape(14.dp)
161.             ) {
162.                 Text(
163.                     text                                = stringResource(R.string.apply_language),
164.                     fontWeight = FontWeight.Bold,
165.                     fontSize = 16.sp
166.                 )
167.             }
168.         }
169.     }
170. }
```

Tabel 22. Source Code FavoritesScreen.kt

```
1.  package com.example.clickanime.ui.favorites
2.
3.  import androidx.compose.foundation.clickable
4.  import androidx.compose.foundation.layout.*
5.  import androidx.compose.foundation.lazy.LazyColumn
6.  import androidx.compose.foundation.lazy.items
7.  import androidx.compose.foundation.shape.RoundedCornerShape
8.  import androidx.compose.material.icons.Icons
9.  import androidx.compose.material.icons.automirrored.filled.ArrowBack
10. import androidx.compose.material.icons.filled.*
11. import androidx.compose.material3.*
12. import androidx.compose.runtime.*
13. import androidx.compose.ui.Alignment
14. import androidx.compose.ui.Modifier
```

```
15. import androidx.compose.ui.draw.clip
16. import androidx.compose.ui.graphics.Color
17. import androidx.compose.ui.layout.ContentScale
18. import androidx.compose.ui.res.stringResource
19. import androidx.compose.ui.text.font.FontWeight
20. import androidx.compose.ui.text.style.TextAlign
21. import androidx.compose.ui.text.style.TextOverflow
22. import androidx.compose.ui.unit.dp
23. import androidx.compose.ui.unit.sp
24. import coil.compose.AsyncImage
25. import com.example.clickanime.R
26. import com.example.clickanime.model.FavoriteMovie
27. import
    com.example.clickanime.viewmodel.MovieViewModel
28.
29. @OptIn(ExperimentalMaterial3Api::class)
30. @Composable
31. fun FavoritesScreen(
32.     viewModel: MovieViewModel,
33.     onMovieClick: (Int) -> Unit,
34.     onBack: () -> Unit
35. ) {
36.     val                  favorites                  by
    viewModel.favorites.collectAsState()
37.
38.     Scaffold(
39.         topBar = {
40.             TopAppBar(
41.                 title = {
42.                     Text(
43.                         text                        =
    "${stringResource(R.string.nav_favorites)}
    (${favorites.size})",
44.                         fontWeight = FontWeight.Bold
```

```
45.                     )
46.                 },
47.                 navigationIcon = {
48.                     IconButton(onClick = onBack) {
49.                         Icon(
50.                             imageVector             = Icons.AutoMirrored.Filled.ArrowBack,
51.                             contentDescription      = stringResource(R.string.back)
52.                         )
53.                     }
54.                 },
55.                 colors                              = TopAppBarDefaults.topAppBarColors(
56.                     containerColor                  = MaterialTheme.colorScheme.primary,
57.                     titleContentColor               = MaterialTheme.colorScheme.onPrimary,
58.                     navigationIconContentColor      = MaterialTheme.colorScheme.onPrimary
59.                 )
60.             )
61.         }
62.     ) { padding ->
63.         if (favorites.isEmpty()) {
64.             EmptyFavoritesContent(modifier          = Modifier.padding(padding))
65.         } else {
66.             LazyColumn(
67.                 modifier = Modifier
68.                     .fillMaxSize()
69.                     .padding(padding),
70.                 contentPadding                      = PaddingValues(vertical = 8.dp)
71.             ) {
```

```
72.                 item {
73.                     Text(
74.                         text                            = stringResource(R.string.favorites_saved_hint),
75.                         style                           = MaterialTheme.typography.bodySmall,
76.                         color                           = MaterialTheme.colorScheme.onSurfaceVariant,
77.                         modifier                        = Modifier.padding(horizontal = 16.dp, vertical = 6.dp)
78.                     )
79.                 }
80.                 items(items = favorites, key = { it.movieId }) { movie ->
81.                     FavoriteItem(
82.                         movie = movie,
83.                         onClick          =          { onMovieClick(movie.movieId) },
84.                         onRemove          =          { viewModel.removeFavorite(movie.movieId) }
85.                     )
86.                 }
87.                 item { Spacer(Modifier.height(16.dp)) }
88.             }
89.         }
90.     }
91. }
92.
93. @Composable
94. private fun EmptyFavoritesContent(modifier: Modifier = Modifier) {
95.     Box(
96.         modifier = modifier.fillMaxSize(),
97.         contentAlignment = Alignment.Center
98.     ) {
```

```
99.         Column(
100.            horizontalAlignment                         =
Alignment.CenterHorizontally,
101.            modifier = Modifier.padding(32.dp)
102.        ) {
103.            Icon(
104.                imageVector                             =
Icons.Default.FavoriteBorder,
105.                contentDescription = null,
106.                modifier = Modifier.size(96.dp),
107.                tint                                    =
MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha
= 0.35f)
108.            )
109.            Spacer(Modifier.height(20.dp))
110.            Text(
111.                text                                    =
stringResource(R.string.favorites_empty_title),
112.                style                                   =
MaterialTheme.typography.titleLarge,
113.                fontWeight = FontWeight.Bold
114.            )
115.            Spacer(Modifier.height(10.dp))
116.            Text(
117.                text                                    =
stringResource(R.string.favorites_empty_desc),
118.                style                                   =
MaterialTheme.typography.bodyMedium,
119.                color                                   =
MaterialTheme.colorScheme.onSurfaceVariant,
120.                textAlign = TextAlign.Center,
121.                lineHeight = 22.sp
122.            )
123.        }
124.    }
125. }
```

```
126.
127. @Composable
128. private fun FavoriteItem(
129.     movie: FavoriteMovie,
130.     onClick: () -> Unit,
131.     onRemove: () -> Unit
132. ) {
133.     var showDialog by remember { mutableStateOf(false)
     }
134.
135.     if (showDialog) {
136.         AlertDialog(
137.             onDismissRequest = { showDialog = false },
138.             icon                    =                    {
     Icon(Icons.Default.DeleteOutline,  contentDescription
     = null) },
139.             title                   =                    {
     Text(stringResource(R.string.remove_favorite)) },
140.             text = {
141.                 Text(
142.
     String.format(stringResource(R.string.remove_favorite
     _confirm), movie.title)
143.                 )
144.             },
145.             confirmButton = {
146.                 TextButton(onClick = {
147.                     onRemove()
148.                     showDialog = false
149.                 }) {
150.                     Text(
151.
     stringResource(R.string.remove),
152.                         color                           =
     MaterialTheme.colorScheme.error
```

```
153.                     )
154.                 }
155.             },
156.             dismissButton = {
157.                 TextButton(onClick = { showDialog = false }) {
158.                 Text(stringResource(R.string.cancel))
159.                 }
160.             }
161.         )
162.     }
163.
164.     Card(
165.         modifier = Modifier
166.             .fillMaxWidth()
167.             .padding(horizontal = 16.dp, vertical = 5.dp)
168.             .clickable { onClick() },
169.         shape = RoundedCornerShape(14.dp),
170.         elevation = CardDefaults.cardElevation(3.dp),
171.         colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)
172.     ) {
173.         Row(
174.             modifier = Modifier
175.                 .fillMaxWidth()
176.                 .padding(10.dp),
177.             verticalAlignment = Alignment.CenterVertically
178.         ) {
179.             AsyncImage(
180.                 model = movie.posterUrl,
181.                 contentDescription = movie.title,
```

```
182.                 contentScale = ContentScale.Crop,
183.                 modifier = Modifier
184.                     .width(68.dp)
185.                     .height(100.dp)
186.                     .clip(RoundedCornerShape(10.dp))
187.             )
188.
189.             Spacer(Modifier.width(12.dp))
190.
191.             Column(Modifier.weight(1f)) {
192.                 Text(
193.                     text = movie.title,
194.                     fontWeight = FontWeight.Bold,
195.                     fontSize = 14.sp,
196.                     maxLines = 2,
197.                     overflow = TextOverflow.Ellipsis
198.                 )
199.                 Spacer(Modifier.height(5.dp))
200.                 Row(verticalAlignment                 =
    Alignment.CenterVertically) {
201.                     Icon(
202.                         Icons.Default.Star, null,
203.                         tint = Color(0xFFFFD700),
204.                         modifier                      =
    Modifier.size(14.dp)
205.                     )
206.                     Spacer(Modifier.width(3.dp))
207.                     Text(
208.                         text = movie.formattedRating,
209.                         style                         =
    MaterialTheme.typography.bodySmall
210.                     )
211.                     Spacer(Modifier.width(10.dp))
```

```
212.                     Icon(
213.                         Icons.Default.CalendarToday, null,
214.                         tint                         = MaterialTheme.colorScheme.primary,
215.                         modifier                     = Modifier.size(12.dp)
216.                     )
217.                     Spacer(Modifier.width(3.dp))
218.                     Text(
219.                         text = movie.releaseYear,
220.                         style                        = MaterialTheme.typography.bodySmall,
221.                         color                        = MaterialTheme.colorScheme.primary
222.                     )
223.                 }
224.                 Spacer(Modifier.height(5.dp))
225.                 Text(
226.                     text = movie.overview,
227.                     style                            = MaterialTheme.typography.bodySmall,
228.                     color                            = MaterialTheme.colorScheme.onSurfaceVariant,
229.                     maxLines = 2,
230.                     overflow = TextOverflow.Ellipsis,
231.                     lineHeight = 18.sp
232.                 )
233.             }
234.
235.             IconButton(
236.                 onClick = { showDialog = true },
237.                 modifier = Modifier.size(40.dp)
238.             ) {
239.                 Icon(
```

```
                    imageVector                    =
Icons.Default.Favorite,
                    contentDescription             =
stringResource(R.string.remove_from_favorites),
                    tint = Color.Red,
                    modifier = Modifier.size(22.dp)
                )
            }
        }
    }
}
```

Tabel 23. Source Code SearchScreen.kt

```
package com.example.clickanime.ui.search

import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import
androidx.compose.foundation.shape.RoundedCornerShape
import
androidx.compose.foundation.text.KeyboardActions
import
androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import
androidx.compose.material.icons.automirrored.filled.A
rrowBack
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
```

```
18. import androidx.compose.ui.graphics.Color
19. import androidx.compose.ui.layout.ContentScale
20. import androidx.compose.ui.platform.LocalFocusManager
21. import androidx.compose.ui.res.stringResource
22. import androidx.compose.ui.text.font.FontWeight
23. import androidx.compose.ui.text.input.ImeAction
24. import androidx.compose.ui.text.style.TextAlign
25. import androidx.compose.ui.text.style.TextOverflow
26. import androidx.compose.ui.unit.dp
27. import androidx.compose.ui.unit.sp
28. import coil.compose.AsyncImage
29. import com.example.clickanime.R
30. import com.example.clickanime.model.Movie
31. import
    com.example.clickanime.viewmodel.MovieSearchUiState
32. import
    com.example.clickanime.viewmodel.MovieViewModel
33.
34. @OptIn(ExperimentalMaterial3Api::class)
35. @Composable
36. fun SearchScreen(
37.     viewModel: MovieViewModel,
38.     onMovieClick: (Int) -> Unit,
39.     onBack: () -> Unit
40. ) {
41.     val     query                                   by
    viewModel.searchQuery.collectAsState()
42.     val                 searchState                 by
    viewModel.searchState.collectAsState()
43.     val focusMgr    = LocalFocusManager.current
44.
45.     DisposableEffect(Unit)     {     onDispose     {
    viewModel.clearSearch() } }
46.
```

```
47.     Scaffold(
48.         topBar = {
49.             TopAppBar(
50.                 navigationIcon = {
51.                     IconButton(onClick = onBack) {
52.                         Icon(Icons.AutoMirrored.Filled.ArrowBack,
53.                         stringResource(R.string.back),         tint         =
                        MaterialTheme.colorScheme.onPrimary)
54.                     }
55.                 },
56.                 title = {
57.                     OutlinedTextField(
58.                         value         = query,
59.                         onValueChange       =       {
                        viewModel.updateSearch(it) },
60.                         placeholder            =    {
                        Text(stringResource(R.string.search_hint),
61.                             style                  =
                        MaterialTheme.typography.bodyMedium) },
62.                         modifier                   =
                        Modifier.fillMaxWidth(),
63.                         singleLine    = true,
64.                         shape                      =
                        RoundedCornerShape(24.dp),
65.                         colors                     =
                        OutlinedTextFieldDefaults.colors(
66.                             focusedBorderColor      =
                            MaterialTheme.colorScheme.onPrimary,
67.                             unfocusedBorderColor    =
                            MaterialTheme.colorScheme.onPrimary.copy(.5f),
68.                             focusedTextColor        =
                            MaterialTheme.colorScheme.onPrimary,
69.                             unfocusedTextColor      =
                            MaterialTheme.colorScheme.onPrimary,
70.                             cursorColor             =
                            MaterialTheme.colorScheme.onPrimary,
```

```
                                focusedPlaceholderColor = MaterialTheme.colorScheme.onPrimary.copy(.7f),
                                unfocusedPlaceholderColor= MaterialTheme.colorScheme.onPrimary.copy(.7f)
                            ),
                            trailingIcon  = {
                                if (query.isNotBlank()) {
                                    IconButton(onClick  = { viewModel.clearSearch() }) {
                                        Icon(Icons.Default.Clear, stringResource(R.string.clear_search),
                                            tint        = MaterialTheme.colorScheme.onPrimary)
                                    }
                                }
                            },
                            keyboardOptions             = KeyboardOptions(imeAction = ImeAction.Search),
                            keyboardActions             = KeyboardActions(onSearch = { focusMgr.clearFocus() })
                        )
                    },
                    colors                              = TopAppBarDefaults.topAppBarColors(
                        containerColor                  = MaterialTheme.colorScheme.primary
                    )
                )
            }
        ) { padding ->
            Box(Modifier.fillMaxSize().padding(padding)) {
                when (val state = searchState) {
                    is MovieSearchUiState.Idle -> {
```

```
95.
EmptyPlaceholder(Icons.Default.Search,
stringResource(R.string.search_empty_hint))
96.                 }
97.                 is MovieSearchUiState.Loading -> {
98.                     Box(Modifier.fillMaxSize(),
contentAlignment = Alignment.Center) {
99.                         CircularProgressIndicator()
100.                    }
101.                }
102.                is MovieSearchUiState.Empty -> {
103.
EmptyPlaceholder(Icons.Default.SearchOff,
104.
"${stringResource(R.string.search_no_results)}
\"$query\"")
105.                }
106.                is MovieSearchUiState.Error -> {
107.                    Box(Modifier.fillMaxSize(),
contentAlignment = Alignment.Center) {
108.                        Column(horizontalAlignment  =
Alignment.CenterHorizontally,       modifier       =
Modifier.padding(24.dp)) {
109.
Icon(Icons.Default.ErrorOutline, null,
110.                                tint               =
MaterialTheme.colorScheme.error,      modifier      =
Modifier.size(56.dp))
111.
Spacer(Modifier.height(12.dp))
112.                            Text(state.message,
textAlign = TextAlign.Center)
113.                        }
114.                    }
115.                }
116.                is MovieSearchUiState.Success -> {
117.                    LazyColumn(contentPadding       =
PaddingValues(vertical = 8.dp)) {
```

```
118.                         item {
119.                             Text(
120.                                 text                =
    "${state.movies.size}
    ${stringResource(R.string.search_results)}
    \"$query\"",
121.                                 style               =
    MaterialTheme.typography.bodySmall,
122.                                 color               =
    MaterialTheme.colorScheme.onSurfaceVariant,
123.                                 modifier            =
    Modifier.padding(horizontal = 16.dp, vertical = 6.dp)
124.                             )
125.                         }
126.                         items(state.movies,  key  =  {
    it.id }) { movie ->
127.                             SearchResultItem(movie  =
    movie, onClick = { onMovieClick(movie.id) })
128.                         }
129.                     }
130.                 }
131.             }
132.         }
133.     }
134. }
135.
136. @Composable
137. private          fun          EmptyPlaceholder(icon:
    androidx.compose.ui.graphics.vector.ImageVector,
    label: String) {
138.     Box(Modifier.fillMaxSize(),  contentAlignment  =
    Alignment.Center) {
139.         Column(horizontalAlignment                  =
    Alignment.CenterHorizontally,       modifier        =
    Modifier.padding(32.dp)) {
140.             Icon(icon,      null,      modifier      =
    Modifier.size(80.dp),
```

```
141.                 tint                                    =
MaterialTheme.colorScheme.onSurfaceVariant.copy(.35f)
)
142.             Spacer(Modifier.height(16.dp))
143.             Text(label,            style            =
MaterialTheme.typography.bodyLarge,
144.                 color                                   =
MaterialTheme.colorScheme.onSurfaceVariant.copy(.6f),
textAlign = TextAlign.Center)
145.         }
146.     }
147. }
148.
149. @Composable
150. private fun SearchResultItem(movie: Movie, onClick: ()
-> Unit) {
151.     Card(
152.         modifier                                        =
Modifier.fillMaxWidth().padding(horizontal  =  16.dp,
vertical = 5.dp).clickable { onClick() },
153.         shape     = RoundedCornerShape(12.dp),
154.         elevation = CardDefaults.cardElevation(2.dp)
155.     ) {
156.         Row(Modifier.fillMaxWidth().padding(10.dp),
verticalAlignment = Alignment.CenterVertically) {
157.             AsyncImage(
158.                 model              = movie.posterUrl,
159.                 contentDescription = movie.title,
160.                 contentScale      = ContentScale.Crop,
161.                 modifier                                =
Modifier.width(55.dp).height(80.dp).clip(RoundedCorne
rShape(8.dp))
162.             )
163.             Spacer(Modifier.width(12.dp))
164.             Column(Modifier.weight(1f)) {
165.                 Text(movie.title,     fontWeight     =
FontWeight.SemiBold, fontSize = 14.sp,
```

```
                    maxLines = 2, overflow = TextOverflow.Ellipsis)
                Spacer(Modifier.height(4.dp))
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Icon(Icons.Default.Star, null, tint = Color(0xFFFFD700), modifier = Modifier.size(13.dp))
                    Spacer(Modifier.width(3.dp))
                    Text(movie.formattedRating, style = MaterialTheme.typography.bodySmall)
                    Spacer(Modifier.width(8.dp))
                    Text(movie.releaseYear, style = MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.primary)
                }
                Spacer(Modifier.height(4.dp))
                Text(movie.overview, style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant,
                    maxLines = 2, overflow = TextOverflow.Ellipsis)
            }
            Icon(Icons.Default.ChevronRight, null, tint = MaterialTheme.colorScheme.onSurfaceVariant)
        }
    }
}
```

Tabel 24. Source Code Settings Screen.kt

```
package com.example.clickanime.ui.settings

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
```

```
5.  import androidx.compose.foundation.shape.RoundedCornerShape
6.  import androidx.compose.foundation.verticalScroll
7.  import androidx.compose.material.icons.Icons
8.  import androidx.compose.material.icons.automirrored.filled.ArrowBack
9.  import androidx.compose.material.icons.filled.*
10. import androidx.compose.material3.*
11. import androidx.compose.runtime.*
12. import androidx.compose.ui.Alignment
13. import androidx.compose.ui.Modifier
14. import androidx.compose.ui.graphics.vector.ImageVector
15. import androidx.compose.ui.res.stringResource
16. import androidx.compose.ui.text.font.FontWeight
17. import androidx.compose.ui.unit.dp
18. import com.example.clickanime.R
19. import com.example.clickanime.data.local.AppPreferences
20. import com.example.clickanime.viewmodel.MovieViewModel
21.
22. @OptIn(ExperimentalMaterial3Api::class)
23. @Composable
24. fun SettingsScreen(
25.     viewModel: MovieViewModel,
26.     preferences: AppPreferences,
27.     onBack: () -> Unit
28. ) {
29.     var notificationsEnabled by remember {
30.         mutableStateOf(preferences.isNotificationsEnabled)
31.     }
32.     var isGridView by remember {
```

```
33.         mutableStateOf(preferences.isGridView)
34.     }
35.     var showClearDialog by remember { mutableStateOf(false) }
36.     var showCacheClearedMsg by remember { mutableStateOf(false) }
37.
38.     if (showClearDialog) {
39.         AlertDialog(
40.             onDismissRequest = { showClearDialog = false },
41.             icon = {
42.                 Icon(Icons.Default.DeleteSweep, contentDescription = null)
43.             },
44.             title = { Text(stringResource(R.string.settings_clear_cache_title)) },
45.             text = { Text(stringResource(R.string.settings_clear_cache_msg)) },
46.             confirmButton = {
47.                 TextButton(onClick = {
48.                     viewModel.clearAllCache()
49.                     showCacheClearedMsg = true
50.                     showClearDialog = false
51.                 }) {
52.                     Text(
53.                         text = stringResource(R.string.settings_clear_cache),
54.                         color = MaterialTheme.colorScheme.error
55.                     )
56.                 }
57.             },
58.             dismissButton = {
```

```
59.                 TextButton(onClick = { showClearDialog = false }) {
60.                     Text(stringResource(R.string.cancel))
61.                 }
62.             }
63.         )
64.     }
65.
66.     Scaffold(
67.         topBar = {
68.             TopAppBar(
69.                 title = {
70.                     Text(
71.                         text = stringResource(R.string.settings_title),
72.                         fontWeight = FontWeight.Bold
73.                     )
74.                 },
75.                 navigationIcon = {
76.                     IconButton(onClick = onBack) {
77.                         Icon(
78.                             imageVector = Icons.AutoMirrored.Filled.ArrowBack,
79.                             contentDescription = stringResource(R.string.back)
80.                         )
81.                     }
82.                 },
83.                 colors = TopAppBarDefaults.topAppBarColors(
84.                     containerColor = MaterialTheme.colorScheme.primary,
85.                     titleContentColor = MaterialTheme.colorScheme.onPrimary,
```

```
86.                     navigationIconContentColor      =
    MaterialTheme.colorScheme.onPrimary
87.                 )
88.             )
89.         }
90.     ) { padding ->
91.         Column(
92.             modifier = Modifier
93.                 .fillMaxSize()
94.                 .padding(padding)
95.
    .verticalScroll(rememberScrollState())
96.                 .padding(16.dp),
97.             verticalArrangement                     =
    Arrangement.spacedBy(4.dp)
98.         ) {
99.
    SectionHeader(stringResource(R.string.settings_app))
100.
101.            ToggleRow(
102.                icon = Icons.Default.Notifications,
103.                title                               =
    stringResource(R.string.settings_notifications),
104.                subtitle                            =
    stringResource(R.string.settings_notifications_sub),
105.                checked = notificationsEnabled,
106.                onCheckedChange = {
107.                    notificationsEnabled = it
108.
    preferences.isNotificationsEnabled = it
109.                }
110.            )
111.
112.            ToggleRow(
113.                icon = Icons.Default.GridView,
```

```
                title                                   = stringResource(R.string.settings_grid_view),
                subtitle                                = stringResource(R.string.settings_grid_view_sub),
                checked = isGridView,
                onCheckedChange = {
                    isGridView = it
                    preferences.isGridView = it
                }
            )

            Spacer(Modifier.height(8.dp))
            HorizontalDivider()
            Spacer(Modifier.height(8.dp))


            SectionHeader(stringResource(R.string.settings_data))

            Card(
                modifier = Modifier.fillMaxWidth(),
                shape = RoundedCornerShape(12.dp),
                colors = CardDefaults.cardColors(
                    containerColor                      = MaterialTheme.colorScheme.surfaceVariant
                )
            ) {
                Column(Modifier.padding(14.dp)) {
                    Row(verticalAlignment               = Alignment.CenterVertically) {
                        Icon(
                            imageVector                 = Icons.Default.Storage,
                            contentDescription          = null,
```

```
                                tint                        =
MaterialTheme.colorScheme.primary,
                                modifier                    =
Modifier.size(20.dp)
                            )
                            Spacer(Modifier.width(10.dp))
                            Column(Modifier.weight(1f)) {
                                Text(
                                    text                    =
stringResource(R.string.settings_cache_strategy),
                                    fontWeight              =
FontWeight.SemiBold,
                                    style                   =
MaterialTheme.typography.bodyMedium
                                )
                                Text(
                                    text                    =
stringResource(R.string.settings_cache_strategy_value
),
                                    style                   =
MaterialTheme.typography.bodySmall,
                                    color                   =
MaterialTheme.colorScheme.primary
                                )
                            }
                        }
                        Spacer(Modifier.height(8.dp))
                        Text(
                            text                            =
stringResource(R.string.settings_cache_desc),
                            style                           =
MaterialTheme.typography.bodySmall,
                            color                           =
MaterialTheme.colorScheme.onSurfaceVariant,
                            lineHeight                      =
MaterialTheme.typography.bodySmall.lineHeight
                        )
```

```
165.                 }
166.             }
167.
168.             Spacer(Modifier.height(8.dp))
169.
170.             OutlinedButton(
171.                 onClick = { showClearDialog = true },
172.                 modifier = Modifier.fillMaxWidth(),
173.                 shape = RoundedCornerShape(10.dp),
174.                 colors                                  =
ButtonDefaults.outlinedButtonColors(
175.                     contentColor                        =
MaterialTheme.colorScheme.error
176.                 )
177.             ) {
178.                 Icon(
179.                     imageVector                         =
Icons.Default.DeleteSweep,
180.                     contentDescription = null,
181.                     modifier = Modifier.size(18.dp)
182.                 )
183.                 Spacer(Modifier.width(8.dp))
184.                 Text(
185.                     text                                =
stringResource(R.string.settings_clear_cache),
186.                     fontWeight = FontWeight.Medium
187.                 )
188.             }
189.
190.             if (showCacheClearedMsg) {
191.                 Spacer(Modifier.height(4.dp))
192.                 Row(verticalAlignment                   =
Alignment.CenterVertically) {
193.                     Icon(
```

```
194.                         imageVector                 = Icons.Default.CheckCircle,
195.                         contentDescription = null,
196.                         tint                        = MaterialTheme.colorScheme.primary,
197.                         modifier                    = Modifier.size(16.dp)
198.                     )
199.                     Spacer(Modifier.width(6.dp))
200.                     Text(
201.                         text                        = stringResource(R.string.settings_cache_cleared),
202.                         style                       = MaterialTheme.typography.bodySmall,
203.                         color                       = MaterialTheme.colorScheme.primary
204.                     )
205.                 }
206.             }
207.
208.             Spacer(Modifier.height(8.dp))
209.             HorizontalDivider()
210.             Spacer(Modifier.height(8.dp))
211.
212.             SectionHeader(stringResource(R.string.settings_about))
213.
214.             Card(
215.                 modifier = Modifier.fillMaxWidth(),
216.                 shape = RoundedCornerShape(12.dp),
217.                 colors = CardDefaults.cardColors(
218.                     containerColor              = MaterialTheme.colorScheme.surfaceVariant
219.                 )
220.             ) {
```

```
221.                 Column(
222.                     Modifier.padding(14.dp),
223.                     verticalArrangement             =
    Arrangement.spacedBy(4.dp)
224.                 ) {
225.                     Text(
226.                         text                        =
    stringResource(R.string.settings_app_label),
227.                         fontWeight = FontWeight.Bold,
228.                         style                       =
    MaterialTheme.typography.bodyMedium
229.                     )
230.                     HorizontalDivider(
231.                         modifier                    =
    Modifier.padding(vertical = 4.dp),
232.                         color                       =
    MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha
    = 0.2f)
233.                     )
234.                     AboutRow(Icons.Default.School,
    stringResource(R.string.settings_about_module))
235.                     AboutRow(Icons.Default.Movie,
    stringResource(R.string.settings_about_tmdb))
236.
    AboutRow(Icons.Default.NetworkCheck,
    stringResource(R.string.settings_about_networking))
237.                     AboutRow(Icons.Default.Storage,
    stringResource(R.string.settings_about_storage))
238.                 }
239.             }
240.
241.             Spacer(Modifier.height(24.dp))
242.         }
243.     }
244. }
245.
246. @Composable
```

```
247. private fun SectionHeader(title: String) {
248.     Text(
249.         text = title,
250.         style = MaterialTheme.typography.labelLarge,
251.         color = MaterialTheme.colorScheme.primary,
252.         fontWeight = FontWeight.Bold,
253.         modifier = Modifier.padding(top = 4.dp, bottom
     = 6.dp)
254.     )
255. }
256.
257. @Composable
258. private fun ToggleRow(
259.     icon: ImageVector,
260.     title: String,
261.     subtitle: String,
262.     checked: Boolean,
263.     onCheckedChange: (Boolean) -> Unit
264. ) {
265.     Row(
266.         modifier = Modifier
267.             .fillMaxWidth()
268.             .padding(vertical = 6.dp),
269.         verticalAlignment                                =
     Alignment.CenterVertically
270.     ) {
271.         Icon(
272.             imageVector = icon,
273.             contentDescription = null,
274.             tint = MaterialTheme.colorScheme.primary,
275.             modifier = Modifier.size(22.dp)
276.         )
277.         Spacer(Modifier.width(12.dp))
```

```
278.         Column(Modifier.weight(1f)) {
279.             Text(
280.                 text = title,
281.                 style                                   =
MaterialTheme.typography.bodyMedium,
282.                 fontWeight = FontWeight.Medium
283.             )
284.             Text(
285.                 text = subtitle,
286.                 style                                   =
MaterialTheme.typography.bodySmall,
287.                 color                                   =
MaterialTheme.colorScheme.onSurfaceVariant
288.             )
289.         }
290.         Switch(
291.             checked = checked,
292.             onCheckedChange = onCheckedChange
293.         )
294.     }
295. }
296.
297. @Composable
298. private fun AboutRow(icon: ImageVector, text: String)
{
299.     Row(
300.         verticalAlignment                               =
Alignment.CenterVertically,
301.         modifier = Modifier.padding(vertical = 2.dp)
302.     ) {
303.         Icon(
304.             imageVector = icon,
305.             contentDescription = null,
306.             tint                                        =
MaterialTheme.colorScheme.onSurfaceVariant,
```

```
307.             modifier = Modifier.size(14.dp)
308.         )
309.         Spacer(Modifier.width(8.dp))
310.         Text(
311.             text = text,
312.             style                                          =
MaterialTheme.typography.bodySmall,
313.             color                                          =
MaterialTheme.colorScheme.onSurfaceVariant
314.         )
315.     }
316. }
```

Tabel 25. Source Code strings.xml English

```
1.  <?xml version="1.0" encoding="utf-8"?>
2.  <resources>
3.      <string name="app_name">ClickAnime</string>
4.      <string name="app_title">Movie Explorer</string>
5.
6.      <!-- Navigation -->
7.      <string name="nav_home">Home</string>
8.      <string name="nav_search">Search</string>
9.      <string name="nav_favorites">Favorites</string>
10.     <string name="nav_settings">Settings</string>
11.
12.     <!-- Home -->
13.     <string name="featured">Featured</string>
14.     <string name="all_movies">All Movies</string>
15.     <string name="category_popular">Popular</string>
16.     <string           name="category_now_playing">Now
Playing</string>
17.     <string             name="category_top_rated">Top
Rated</string>
```

```
18.     <string
name="category_upcoming">Upcoming</string>
19.     <string             name="loading_movies">Loading
movies…</string>
20.     <string name="refresh">Refresh</string>
21.     <string name="retry">Retry</string>
22.     <string   name="failed_to_load">Failed   to   load
movies</string>
23.
24.     <!-- Detail -->
25.     <string name="overview">Overview</string>
26.     <string      name="no_overview">No      overview
available.</string>
27.     <string name="rating">Rating</string>
28.     <string name="runtime">Runtime</string>
29.     <string name="year">Year</string>
30.     <string name="votes">Votes</string>
31.     <string name="box_office">Box Office</string>
32.     <string name="budget">Budget</string>
33.     <string name="revenue">Revenue</string>
34.     <string name="status">Status</string>
35.     <string name="language">Language</string>
36.     <string      name="add_to_favorites">Add      to
Favorites</string>
37.     <string name="remove_from_favorites">Remove from
Favorites</string>
38.
39.     <!-- Search -->
40.     <string                 name="search_hint">Search
movies…</string>
41.     <string name="search_empty_hint">Search for your
favorite movies</string>
42.     <string name="search_no_results">No movies found
for</string>
43.     <string             name="search_results">results
for</string>
```

```
    <string name="clear_search">Clear search</string>

    <!-- Favorites -->
    <string name="favorites_empty_title">No favorites yet</string>
    <string name="favorites_empty_desc">Browse movies and tap ♥ to save them here permanently.</string>
    <string name="favorites_saved_hint">Saved to your device • Stays even offline</string>
    <string name="remove_favorite">Remove Favorite</string>
    <string name="remove_favorite_confirm">Remove \"%s\" from your favorites?</string>
    <string name="remove">Remove</string>
    <string name="cancel">Cancel</string>

    <!-- Settings -->
    <string name="settings_title">Settings</string>
    <string name="settings_app">App Settings</string>
    <string name="settings_notifications">Enable Notifications</string>
    <string name="settings_notifications_sub">Receive updates about new movies</string>
    <string name="settings_grid_view">Grid View</string>
    <string name="settings_grid_view_sub">Display movies in a grid layout</string>
    <string name="settings_data">Data &amp;amp; Cache</string>
    <string name="settings_cache_strategy">Movie Cache Strategy</string>
    <string name="settings_cache_strategy_value">Cache-First with 30-minute TTL</string>
    <string name="settings_cache_desc">Movies are cached locally and refreshed every 30 minutes. Favorites are stored permanently.</string>
    <string name="settings_clear_cache">Clear Movie Cache</string>
```

```
67.     <string name="settings_clear_cache_title">Clear Movie Cache</string>
68.     <string name="settings_clear_cache_msg">This will delete all cached movie data. The app will re-fetch from the internet. Your favorites will NOT be deleted.</string>
69.     <string name="settings_cache_cleared">✓ Cache cleared. Movies will be refreshed from the internet.</string>
70.     <string name="settings_about">About</string>
71.     <string name="settings_app_label">ClickAnime — Movie Explorer</string>
72.     <string name="settings_about_module">Modul 5 — Connect to the Internet</string>
73.     <string name="settings_about_tmdb">Data provided by The Movie Database (TMDB)</string>
74.     <string name="settings_about_networking">Networking: Retrofit + KotlinX Serialization</string>
75.     <string name="settings_about_storage">Storage: Room (cache) + SharedPreferences (settings)</string>
76.
77.     <!-- Language -->
78.     <string name="change_language">Change Language</string>
79.     <string name="select_language">Select Language</string>
80.     <string name="apply_language">Apply</string>
81.     <string name="lang_english">English</string>
82.     <string name="lang_indonesian">Indonesian</string>
83.     <string name="back">Back</string>
84. </resources>
```

Tabel 26. Source Code strings.xml Indonesia

```
1.  <?xml version="1.0" encoding="utf-8"?>
2.  <resources>
3.      <string name="app_name">ClickAnime</string>
```

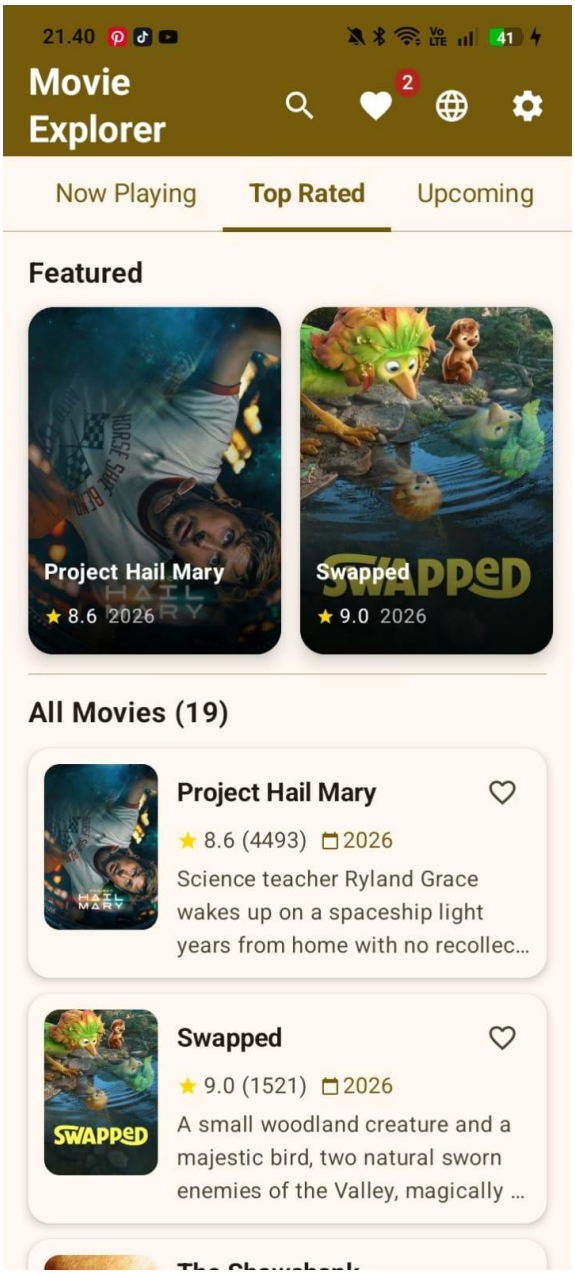
```
4.      <string name="app_title">Penjelajah Film</string>
5.
6.      <string name="nav_home">Beranda</string>
7.      <string name="nav_search">Cari</string>
8.      <string name="nav_favorites">Favorit</string>
9.      <string name="nav_settings">Pengaturan</string>
10.
11.     <string name="featured">Unggulan</string>
12.     <string name="all_movies">Semua Film</string>
13.     <string name="category_popular">Populer</string>
14.     <string name="category_now_playing">Sedang
        Tayang</string>
15.     <string name="category_top_rated">Nilai
        Tertinggi</string>
16.     <string name="category_upcoming">Akan
        Datang</string>
17.     <string name="loading_movies">Memuat
        film…</string>
18.     <string name="refresh">Perbarui</string>
19.     <string name="retry">Coba Lagi</string>
20.     <string name="failed_to_load">Gagal memuat
        film</string>
21.
22.     <string name="overview">Sinopsis</string>
23.     <string name="no_overview">Tidak ada sinopsis
        tersedia.</string>
24.     <string name="rating">Nilai</string>
25.     <string name="runtime">Durasi</string>
26.     <string name="year">Tahun</string>
27.     <string name="votes">Suara</string>
28.     <string name="box_office">Box Office</string>
29.     <string name="budget">Anggaran</string>
30.     <string name="revenue">Pendapatan</string>
31.     <string name="status">Status</string>
32.     <string name="language">Bahasa</string>
```

```
33.     <string name="add_to_favorites">Tambah ke Favorit</string>
34.     <string name="remove_from_favorites">Hapus dari Favorit</string>
35.
36.     <string name="search_hint">Cari film…</string>
37.     <string name="search_empty_hint">Cari film favoritmu</string>
38.     <string name="search_no_results">Tidak ada film ditemukan untuk</string>
39.     <string name="search_results">hasil untuk</string>
40.     <string name="clear_search">Bersihkan pencarian</string>
41.
42.     <string name="favorites_empty_title">Belum ada favorit</string>
43.     <string name="favorites_empty_desc">Jelajahi film dan ketuk ♥ untuk menyimpannya di sini secara permanen.</string>
44.     <string name="favorites_saved_hint">Tersimpan di perangkat • Tetap ada saat offline</string>
45.     <string name="remove_favorite">Hapus Favorit</string>
46.     <string name="remove_favorite_confirm">Hapus \"%s\" dari favorit?</string>
47.     <string name="remove">Hapus</string>
48.     <string name="cancel">Batal</string>
49.
50.     <string name="settings_title">Pengaturan</string>
51.     <string name="settings_app">Pengaturan Aplikasi</string>
52.     <string name="settings_notifications">Aktifkan Notifikasi</string>
53.     <string name="settings_notifications_sub">Terima pembaruan tentang film baru</string>
54.     <string name="settings_grid_view">Tampilan Grid</string>
```

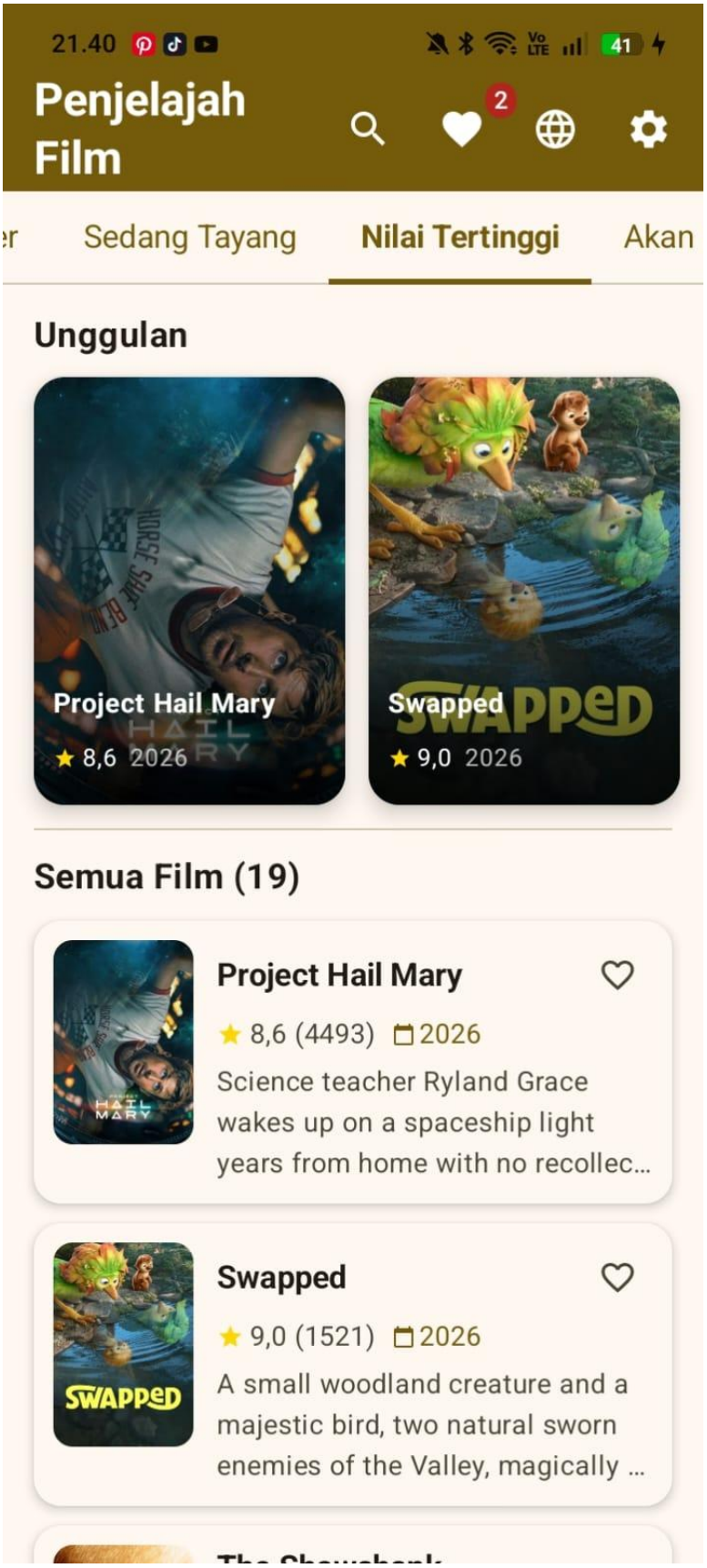
```
55.     <string name="settings_grid_view_sub">Tampilkan film dalam tata letak grid</string>
56.     <string name="settings_data">Data &amp;amp; Cache</string>
57.     <string name="settings_cache_strategy">Strategi Cache Film</string>
58.     <string name="settings_cache_strategy_value">Cache-First dengan TTL 30 menit</string>
59.     <string name="settings_cache_desc">Film di-cache secara lokal dan diperbarui setiap 30 menit. Favorit disimpan permanen.</string>
60.     <string name="settings_clear_cache">Bersihkan Cache Film</string>
61.     <string name="settings_clear_cache_title">Bersihkan Cache Film</string>
62.     <string name="settings_clear_cache_msg">Ini akan menghapus semua data cache film. Aplikasi akan mengambil ulang dari internet. Favorit Anda TIDAK akan dihapus.</string>
63.     <string name="settings_cache_cleared">✓ Cache dibersihkan. Film akan diperbarui dari internet.</string>
64.     <string name="settings_about">Tentang</string>
65.     <string name="settings_app_label">ClickAnime — Penjelajah Film</string>
66.     <string name="settings_about_module">Modul 5 — Connect to the Internet</string>
67.     <string name="settings_about_tmdb">Data disediakan oleh The Movie Database (TMDB)</string>
68.     <string name="settings_about_networking">Jaringan: Retrofit + KotlinX Serialization</string>
69.     <string name="settings_about_storage">Penyimpanan: Room (cache) + SharedPreferences (pengaturan)</string>
70.
71.     <string name="change_language">Ganti Bahasa</string>
72.     <string name="select_language">Pilih Bahasa</string>
```

```
73.     <string name="apply_language">Terapkan</string>
74.     <string                 name="lang_english">Bahasa
Inggris</string>
75.     <string              name="lang_indonesian">Bahasa
Indonesia</string>
76.     <string name="back">Kembali</string>
77. </resources>
```

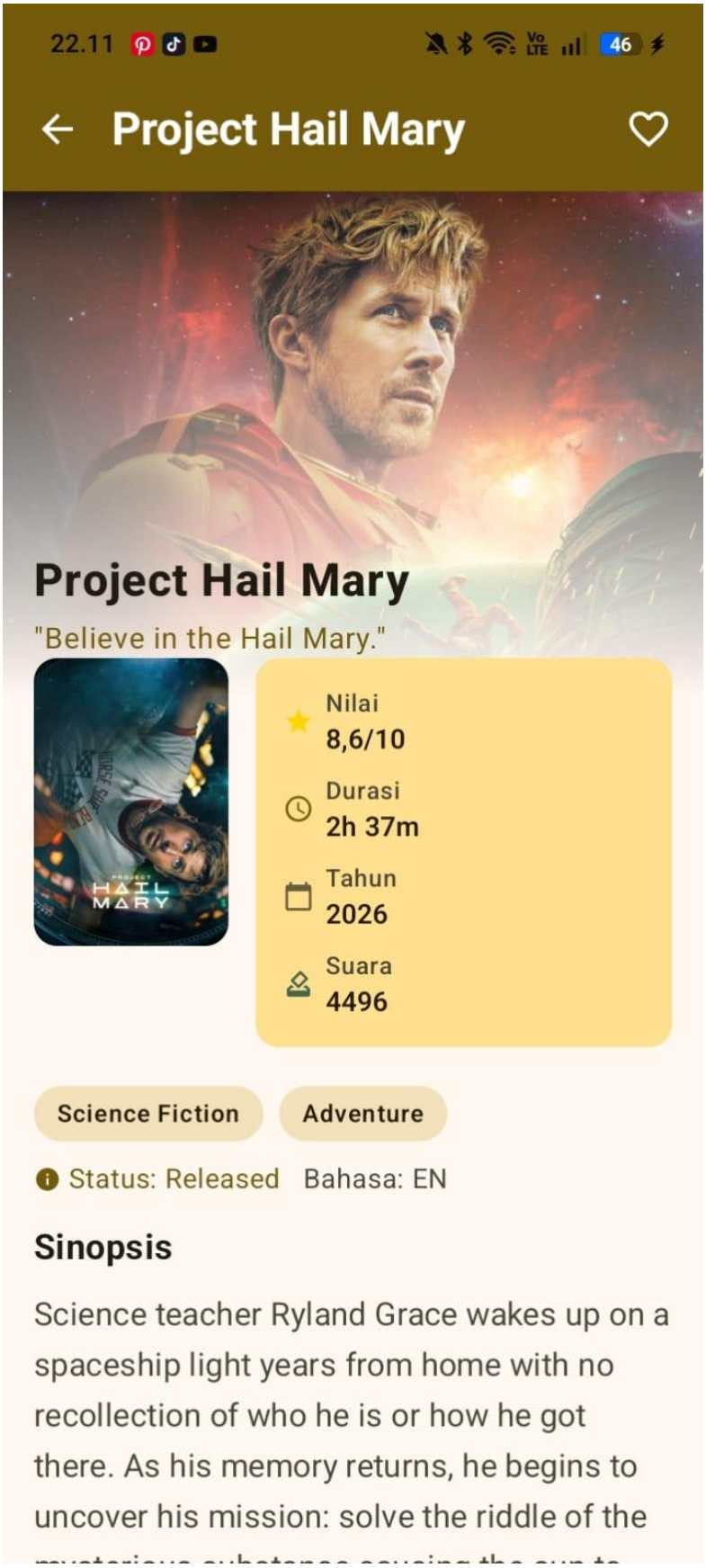
B. OUTPUT CODE



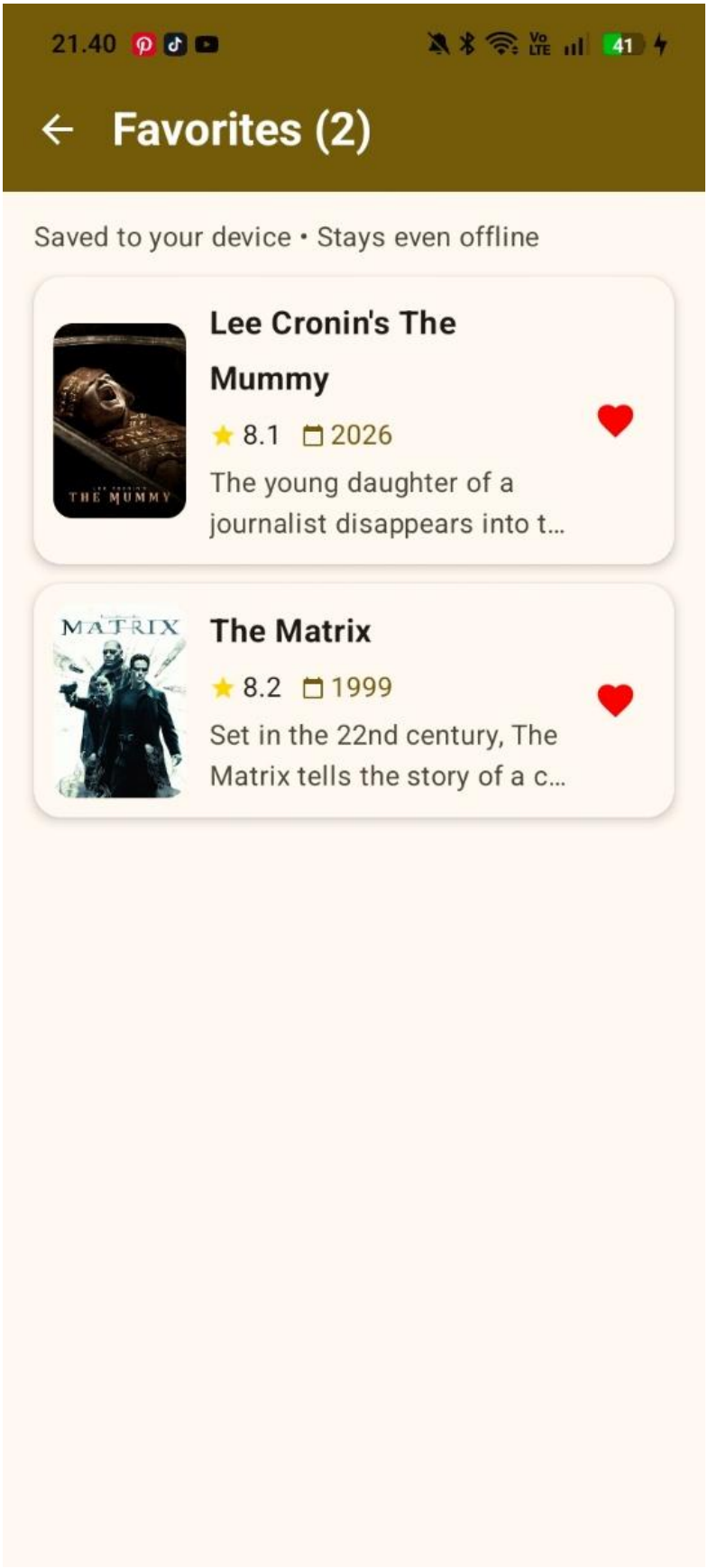
Gambar 1. Homepage English



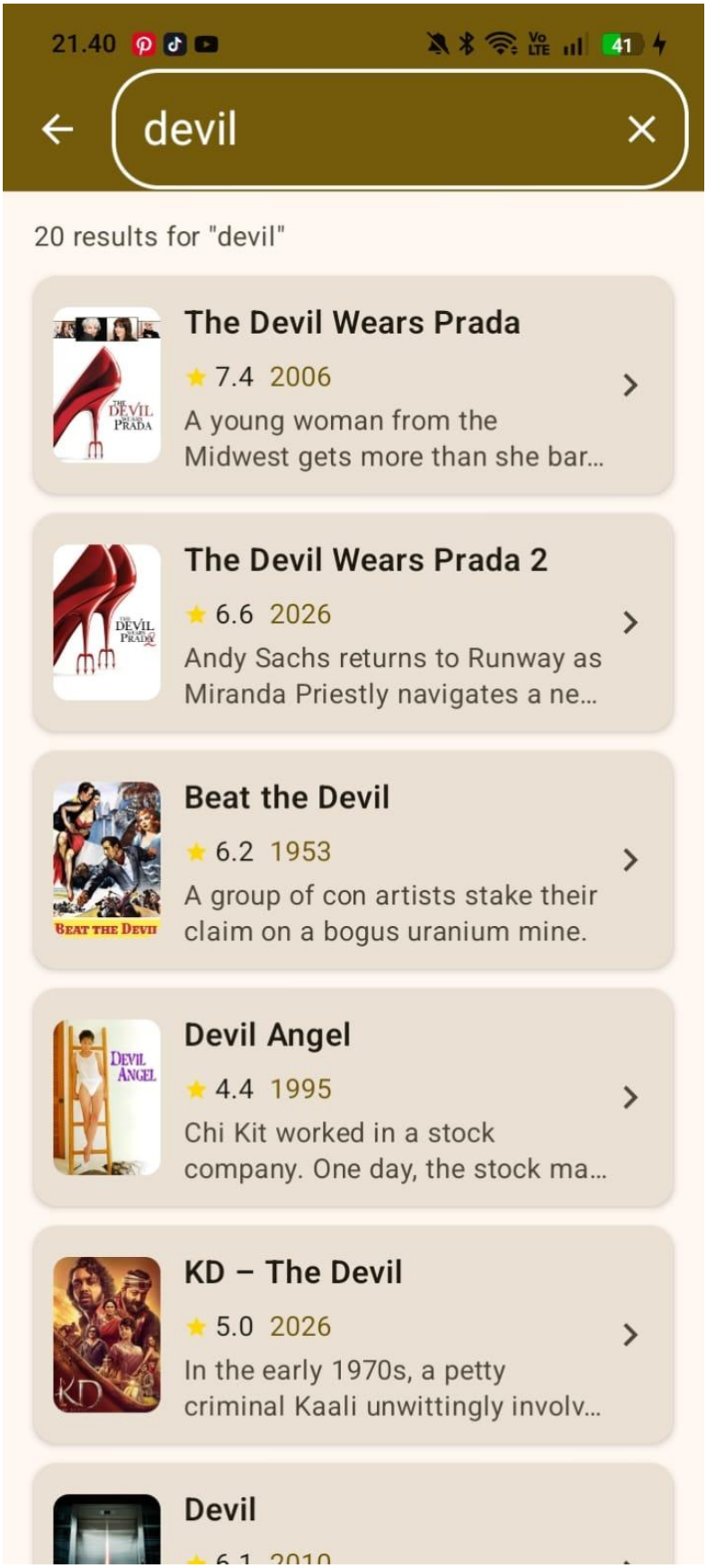
Gambar 2. Home Page Indonesia



Gambar 3. Detail Page Indonesia



Gambar 4. Favorites Page



Gambar 5. Search Page

C. PEMBAHASAN

MainActivity.kt

MainActivity berperan sebagai titik masuk utama (entry point) aplikasi. Kelas ini mengatur tampilan ke layar penuh (edge-to-edge) dan mengelola pengaturan bahasa (Locale) dengan mengambil data dari SharedPreferences sebelum Context di attach. Di dalam fungsi onCreate, aktivitas ini membangun antarmuka menggunakan Jetpack Compose, menginisialisasi MovieViewModel dari *factory* yang ada di AppContainer, dan meneruskan kontrol navigasi utama ke NavGraph.

MovieApp.kt

MovieApp mewarisi kelas Application dan merupakan titik inisialisasi awal sebelum komponen aplikasi lain dijalankan. File ini berfungsi sebagai tempat inisialisasi global untuk AppContainer yang mengatur *Dependency Injection* manual. Selain itu, modul ini juga mengonfigurasi library Timber untuk sistem pencatatan log (logging) otomatis yang hanya aktif pada mode debug, sehingga memudahkan pelacakan tanpa membebani versi rilis.

MovieDao.kt

MovieDao merupakan *Data Access Object* (DAO) untuk library Room yang mengatur operasi database lokal. File ini mendefinisikan kueri-kueri SQL untuk mengelola data cache film, detail film, dan daftar film favorit. Fungsi di dalamnya memanfaatkan *Coroutines* dan *Flow* secara asinkron untuk operasi seperti menyimpan (insert), mengambil data berdasarkan kategori, mencari data, hingga membersihkan cache yang sudah kedaluwarsa.

MovieEntity.kt

MovieEntity mendefinisikan representasi tabel di dalam database Room melalui anotasi @Entity. File ini berisi struktur *Data Class* untuk film umum (movies), detail film (movie_details), dan film favorit (favorite_movies). Entitas-entitas ini bertindak sebagai skema basis data lokal yang menyimpan cache dari API sehingga aplikasi dapat menampilkan data kembali meski tanpa koneksi internet.

AppPreferences.kt

AppPreferences adalah kelas utilitas (wrapper) yang mengelola penyimpanan persisten data ringan menggunakan SharedPreferences. File ini menangani status pengaturan pengguna yang sifatnya global, seperti pilihan bahasa (language), preferensi tampilan antarmuka (grid view), status notifikasi, serta menyimpan state terakhir dari kategori film yang diakses agar aplikasi mengingat pilihan pengguna di sesi berikutnya.

MovieDatabase.kt

MovieDatabase adalah kelas abstrak turunan RoomDatabase yang berfungsi sebagai pengelola utama database lokal. File ini mendefinisikan seluruh entitas tabel dan menyambungkannya dengan DAO yang relevan. Kelas ini mengimplementasikan pola *Singleton* untuk memastikan hanya ada satu instance database yang dibuat dan beroperasi dalam satu memori berjalan untuk menghindari bentrok pada koneksi SQLite.

MovieMapper.kt

MovieMapper berfungsi sebagai pemetaan (mapper) arsitektur data. File ini berisi *extension functions* untuk mengubah objek dari satu lapisan ke lapisan lain. Ia merombak data *Data Transfer Object* (DTO) hasil *response* API menjadi *Entity* untuk disimpan di database lokal Room, dan kemudian mengubah entitas tersebut menjadi model *Domain* yang bersih untuk siap disajikan ke antarmuka pengguna (UI).

TmdbResponse.kt

TmdbResponse menampung struktur *Data Class* khusus untuk menerjemahkan JSON dari TMDB API. Menggunakan anotasi dari *KotlinX Serialization* (@SerialName), file ini memastikan kunci JSON di *parsing* dengan presisi ke variabel Kotlin. Selain itu, terdapat *sealed class* ApiResponse untuk membungkus hasil permintaan jaringan menjadi tiga state fundamental: *Success*, *Error*, dan *Exception*.

ApiHandler.kt

ApiHandler menyediakan fungsi utilitas safeApiCall yang dirancang untuk menjalankan pemanggilan Retrofit API dalam blok *try-catch* secara aman. File ini menstandarkan semua respons jaringan dan *parsing error*, sehingga alih-alih melempar interupsi secara langsung yang membuat aplikasi mogok (*crash*), fungsi ini akan selalu mengembalikan objek tipe ApiResponse yang terprediksi.

NetworkClient.kt

NetworkClient mengonfigurasi klien HTTP dan Retrofit. Di dalamnya terdapat *OkHttpClient* yang diinjeksi dengan *Interceptor* khusus untuk selalu menyematkan *API Key* TMDB pada *header* setiap *request*. File ini juga mengintegrasikan pemformatan respons JSON berbasis *KotlinX Serialization* dan *logging interceptor* untuk menampilkan rekam jejak lalu lintas jaringan pada Android Studio saat mode debug.

TmdbApiService.kt

TmdbApiService adalah antarmuka (interface) khusus Retrofit yang memetakan URL dan *endpoint* API dari The Movie Database. File ini menggunakan anotasi HTTP seperti @GET untuk menarik daftar film dari berbagai kategori

(populer, *now playing*, *top rated*), mencari film spesifik, dan memanggil rincian lengkap profil satu film secara asinkron (menggunakan parameter *suspend*).

MovieRepository.kt

MovieRepository bertindak sebagai *Single Source of Truth* pengelola aliran data dengan menerapkan *Caching Strategy*. File ini terlebih dahulu mengecek waktu kedaluwarsa cache di Room; jika valid, aplikasi hanya memuat data lokal agar cepat (*offline-first*). Apabila kedaluwarsa, *Repository* ini akan menarik respons terbaru dari *NetworkClient*, menyimpan perubahannya di *Database*, lalu memancarkannya kembali ke *ViewModel* menggunakan Flow.

AppContainer.kt

AppContainer menerapkan arsitektur injeksi dependensi manual untuk level aplikasi. File ini membuat, mengelola, dan membagikan satu instansi tunggal (*singleton*) komponen inti yang berat (seperti koneksi database, *Retrofit*, dan *Repository*) ke kelas-kelas lain secara terpusat, sehingga mencegah memori bengkak akibat pembuatan objek (instansiasi) yang berulang-ulang di berbagai layar UI.

Movie.kt

Movie.kt merupakan definisi model tingkat *Domain* yang murni dan bersih dari *framework* API atau Database. Model data (seperti Movie dan MovieDetail) didesain sangat spesifik untuk kebutuhan UI, termasuk mengimplementasikan properti komputasi (*computed properties*) yang langsung merangkai URL format poster, mengambil potongan tahun rilis, dan melakukan penyesuaian string rating secara otomatis.

MovieUiState.kt

MovieUiState menggunakan hierarki kelas tertutup (*sealed classes*) untuk memodelkan berbagai kemungkinan status pada halaman UI (Loading, Success, Error, atau Idle). File ini menjamin bahwa komposisi di *Jetpack Compose* bersifat prediktif dan hanya merender antarmuka berdasarkan kepastian data yang dilempar, menjaga UI tidak me render status yang tumpeng tindih.

MovieViewModel.kt

MovieViewModel menjadi jembatan utama logika interaksi di sisi klien. Kelas ini menampung data UI reaktif (StateFlow) terkait detail, kategori, daftar pencarian, maupun koleksi favorit. Semua operasi asinkron dipanggil melalui *viewModelScope* agar tidak memblokir antarmuka, sekaligus mengatur delegasi ke MovieRepository (seperti saat kategori di tap, pencarian diketik, hingga interaksi simpan favorit).

NavGraph.kt

NavGraph menampung topologi perpindahan setiap antarmuka (*Screen*) dalam satu kendali komposisi via *NavHost*. Skema ini menarik setiap layar (seperti HomeScreen, DetailScreen, SettingsScreen) serta mendelegasikan perpindahan rute dan penangkapan argumen seperti movieId. Sistem fungsi navigasi dilempar sebagai parameter *lambda* untuk meminimalisasi ketergantungan antar komponen visual langsung.

Screen.kt

Screen membentuk skema objek berkonsep *sealed class* yang mewakili semua identitas rute (rute dasar berupa teks string). File ini mengeliminasi penulisan label teks rute yang rentan *typo* (*hardcoded strings*), dan mencakup fungsi helper (createRoute) yang secara sistematis menyisipkan ID film spesifik ketika aplikasi perlu melompat dari daftar ke halaman DetailScreen.

HomeScreen.kt

HomeScreen bertugas melukis visual papan kemudi depan dengan menanamkan tata letak *Scaffold*. Halaman ini memonitor status *ViewModel* via collectAsState() yang segera merender daftar film vertikal atau korsel film horisontal (carousel). Terdapat juga integrasi navigasi interaktif (*TopAppBar*) pada *header* yang memberikan akses mulus untuk pencarian, favorit, maupun menuju bagian panel pengaturan.

DetailScreen.kt

DetailScreen menciptakan komposisi antarmuka yang mengkonsumsi *StateFlow* detail profil film secara utuh. Ketika sebuah rute movieId ditangkap, file ini akan memicu *ViewModel* mengunduh data TMDB secara terperinci. Elemen UI menampilkan lapisan poster grafis bertumpuk (*backdrop*), durasi tayang, status rilis, dan ikhtisar sinopsis lengkap yang dilengkapi *Floating Action Button* berfitur interaksi favorit *real-time*.

LanguageScreen.kt

LanguageScreen merepresentasikan elemen UI tempat pengguna menentukan prioritas bahasa lokalisasi. Aksi klik dalam *layout* ini akan mengeksekusi parameter *callback* ke arah komponen induk agar memutakhirkan preferensi basis AppPreferences dan segera memulai ulang paksa struktur aktivitas dasar, sehingga *resource* aplikasi terganti sepenuhnya di tingkat sistem.

FavoritesScreen.kt

FavoritesScreen merender area layar perpustakaan pribadi yang diamati sistem dari status Flow koleksi *database* lokal (Room). Layar ini menjamin visual list disinkronisasi langsung jika pengguna mencabut status hati (*unfavorite*). UI menangani logika mandiri untuk menampilkan pesan indikator kosong ketika basis data *favorite* belum memiliki satu baris entri pun.

SearchScreen.kt

SearchScreen memberikan wadah kolom pencarian (input form) dan menangani aliran teks dinamis *real-time*. Dengan pola *debounce* pada *ViewModel* untuk menjaga agar laju pengetikan (query) tidak membebani limit API, halaman ini langsung merespons barisan hasil pencarian spesifik, sembari menukar *state* tampilan seketika menjadi ikon muat ulang (*loading spinner*) hingga sukses melukis rentetan *card* film.

SettingsScreen.kt

SettingsScreen adalah panel UI minimalis yang menyajikan fitur manajemen aplikasi tingkat lanjut kepada pengguna, yang dirancang menggunakan *modifier Jetpack Compose* yang responsif. Tampilan ini menampilkan informasi modul/developer, sekaligus memiliki opsi pembersihan file sampah (Clear Cache) yang saat ditekan akan menstimulasi *Repository* via *ViewModel* untuk merombak serta menghapus histori tabel *database*.

strings.xml English

strings.xml English berperan sebagai fondasi *resource* tekstual (hardcode text centralization) standar dalam lingkup sistem Android. Semua istilah tombol, dialog, deskripsi film, dan *placeholder* error digabung di satu repositori *XML*, sehingga kode Kotlin murni bebas dari *string* mentah dan pemeliharaan kosakata menjadi lebih rapi di satu titik modifikasi.

strings.xml Indonesia

strings.xml Indonesia adalah implementasi dukungan multibahasa (lokalisasi i18n). Berkas sumber alternatif (berada di jalur folder values-id) ini menjabarkan seluruh terjemahan yang sinkron dengan *file string* Inggris. Ketika modul MainActivity menyuntikkan *Locale* bahasa Indonesia ke *base context*, aplikasi seketika mendeteksi dan menggunakan seluruh matriks bahasa di sini untuk merombak UI.

TAUTAN GIT

https://github.com/ERISCHA24/PRAKTIKUMMOBILE.git